



NÁZEV PŘÍKLADNÉ ZÁVĚREČNÉ PRÁCE

Bc. Jiří Nebozíz

Diplomová práce
Fakulta informačních technologií
České vysoké učení technické v Praze
Katedra teoretické informatiky
Studijní program: Informatika
Specializace: Softwarové inženýrství
Vedoucí: doc. Ing. Damien Zlo, Ph.D.
1. února 2026

Replace the contents of this file with official assignment.
Místo tohoto souboru sem patří list se zadáním závěrečné práce.

České vysoké učení technické v Praze

Fakulta informačních technologií

© 2021 Bc. Jiří Nebozíz. Všechna práva vyhrazena.

Tato práce vznikla jako školní dílo na Českém vysokém učení technickém v Praze, Fakultě informačních technologií. Práce je chráněna právními předpisy a mezinárodními úmluvami o právu autorském a právech souvisejících s právem autorským. K jejímu užití, s výjimkou bezúplatných zákonných licencí a nad rámec oprávnění uvedených v Prohlášení, je nezbytný souhlas autora.

Odkaz na tuto práci: Nebozíz Jiří. *Název příkladné závěrečné práce*. Diplomová práce.

České vysoké učení technické v Praze, Fakulta informačních technologií, 2021.

Chtěl bych poděkovat především sit amet, consectetur adipiscing elit. Curabitur sagittis hendrerit ante. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Cras pede libero, dapibus nec, pretium sit amet, tempor quis. Sed vel lectus. Donec odio tempus molestie, porttitor ut, iaculis quis, sem. Suspendisse sagittis ultrices augue.

Prohlášení

FILL IN ACCORDING TO THE INSTRUCTIONS. VYPLŇTE V SOULADU S POKYNY. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Curabitur sagittis hendrerit ante. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Cras pede libero, dapibus nec, pretium sit amet, tempor quis. Sed vel lectus. Donec odio tempus molestie, porttitor ut, iaculis quis, sem. Suspendisse sagittis ultrices augue. Donec ipsum massa, ullamcorper in, auctor et, scelerisque sed, est. In sem justo, commodo ut, suscipit at, pharetra vitae, orci. Pellentesque pretium lectus id turpis.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Curabitur sagittis hendrerit ante. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Cras pede libero, dapibus nec, pretium sit amet, tempor quis. Sed vel lectus. Donec odio tempus molestie, porttitor ut, iaculis quis, sem. Suspendisse sagittis ultrices augue. Donec ipsum massa, ullamcorper in, auctor et, scelerisque sed, est. In sem justo, commodo ut, suscipit at, pharetra vitae, orci. Pellentesque pretium lectus id turpis.

V Praze dne 1. února 2026

Abstrakt

Fill in the abstract of this thesis in Czech. Lorem ipsum dolor sit amet. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Cras pede libero, dapibus nec, pretium sit amet, tempor quis. Sed vel lectus. Donec odio tempus molestie, porttitor ut, iaculis quis, sem. Suspendisse sagittis ultrices augue.

Klíčová slova enter, comma, separated, list, of, keywords, in, CZECH

Abstract

Fill in the abstract of this thesis in English. Lorem ipsum dolor sit amet. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Cras pede libero, dapibus nec, pretium sit amet, tempor quis. Sed vel lectus. Donec odio tempus molestie, porttitor ut, iaculis quis, sem. Suspendisse sagittis ultrices augue.

Keywords enter, comma, separated, list, of, keywords, in, ENGLISH

Obsah

Úvod	1
1 Analýza	3
1.1 Analýza stávající aplikace	3
1.1.1 Analýza kódu a architektury	3
1.1.2 Analýza existujících funkcí	4
1.1.3 Návrh uživatelského rozhraní	5
1.2 Analýza existujících řešení	5
1.2.1 Mimo	5
1.2.2 Codecademy	9
1.2.3 Sololearn	13
1.2.4 Scrimba	16
1.2.5 Závěr analýzy	19
1.3 Analýza požadavků	22
1.3.1 Funkční požadavky	23
1.3.2 Nefunkční požadavky	36
2 Návrh	39
2.1 Návrh vzhledu uživatelského rozhraní	39
2.1.1 Principy návrhu	40
2.1.2 Výběr palety barev	41
2.1.3 Typografie	46
2.1.4 Ikony a ilustrace	48
2.1.5 Animace a zvukové efekty	48
2.2 Návrh prototypu uživatelského rozhraní	49
2.2.1 Tvorba prototypu	49
2.2.2 Responsivita	50
2.2.3 Testování prototypu	51
2.3 Návrh funkcí aplikace	52
2.3.1 Popis aktérů	53
2.3.2 Specifikace případů užití	53
2.3.3 Specifikace funkcí pomocí jazyka Gherkin	54
3 Implementace	57
3.1 Postup při realizaci projektu	57
3.1.1 Příprava implementace	57

Obsah	viii
4 Testování	59
Závěr	60
A Nějaká příloha	61
Obsah příloh	66

Seznam obrázků

2.1	Volba primární barvy	42
2.2	Volba barev pro indikaci stavů	44
2.3	Volba barev pozadí a povrchů	45
2.4	Ukázka návrhu hlavní stránky aplikace	51
2.5	Ukázka návrhu cvičení programování	51
2.6	Ukázka návrhu žebříčku uživatelů	52
2.7	Ukázka návrhu uživatelského profilu	52

Seznam tabulek

Seznam výpisů kódu

1	Ukázka specifikace funkce pomocí jazyka Gherkin	56
---	---	----

Seznam zkratek

WCAG 2.2	Web Content Accessibility Guidelines 2.2
TDD	Test-Driven Development
BDD	Behavior-Driven Development
API	Application Programming Interface
UI	User Interface
XP	Experience Points

Úvod

V posledních letech dochází k výrazným změnám v oblasti vzdělávání, a to zejména díky rozvoji nových technologií a metod učení. Stále větší popularitu získává mikrolearning, tedy učení prostřednictvím krátkých a efektivních lekcí. Ten však zatím není ve většině oborů dostatečně rozšířen. S nástupem umělé inteligence, zejména velkých jazykových modelů, se také zásadně mění možnosti, jak efektivně a personalizovaně přistupovat k učení. Tradiční systémy, jako jsou například vzdělávací kurzy nebo aplikace využívající spaced repetition algoritmy, často nedokáží držet krok s tímto rychlým vývojem. V moderních aplikacích je navíc kladen čím dál tím větší důraz na uživatelskou zkušenost, která stále u mnoha vzdělávacích systémů není na dostatečně vysoké úrovni.

Na základě těchto skutečností jsme se rozhodli s kolegou Bc. Jakubem Vondráčkem vytvořit moderní vzdělávací systém ve formě webové aplikace, který bude reflektovat aktuální změny v oblasti vzdělávání. Hlavním cílem této aplikace je propojit efektivní metody učení, s kvalitní uživatelskou zkušeností a možnostmi, které poskytuje umělá inteligence. V rámci této práce bude aplikace zaměřena především na výuku programování a softwarového inženýrství. Bude ovšem navržena tak, aby byla snadno rozšiřitelná a využitelná v různých oblastech vzdělávání. Důraz bude kladen na zábavnost, efektivitu a personalizaci učení, které jsou klíčové pro dlouhodobou motivaci uživatelů.

Systém bude rozšířením již existující aplikace na výuku jazyků, kterou jsme společně s kolegou Bc. Janem Hlaváčem vytvořili v rámci našich bakalářských prací. Díky tomu bude možné projekt stavět na již vybudovaných základech a zaměřit se tak na implementaci pokročilejších funkcionalit.

Projekt byl rozdělen do dvou diplomových prací. Tato práce bude zaměřena na podrobnější analýzu požadavků a následný vývoj frontendové části aplikace. Diplomová práce kolegy Bc. Jakuba Vondráčka se pak bude převážně zabývat vývojem backendové části této aplikace.

Z hlavního cíle vytvoření vzdělávací aplikace přirozeně vyplývají dílčí cíle, které reflektují jednotlivé fáze životního cyklu softwarového projektu. Prvním cílem je provést analýzu existující aplikace na výuku jazyků a sepsat poža-

daty, které budou sloužit jako podklad pro vytvoření nové aplikace. V rámci analýzy bude také provedena analýza konkurenčních řešení v oboru výuky softwarového inženýrství. Druhým cílem je sepsat specifikaci nových funkcionalit a navrhnout jejich uživatelské rozhraní. Součástí tohoto cíle je také provedení aktualizace vzhledu existujícího uživatelského rozhraní. Třetím cílem je provedení samotné implementace funkcionalit a popsání použitých postupů a nově zavedených technologií. Čtvrtým cílem je zvolit a provést vhodné formy testování nových funkcionalit. Posledním cílem je pak zhodnocení celého řešení a navrnutí možných úprav do budoucna.

Práce bude zaměřena výhradně na vývoj aplikace a nebude zahrnovat vytváření samotného obsahu, jako například tvorbu konkrétních kurzů, lekcí nebo cvičení.

Výsledná aplikace bude sloužit studentům a samoukům, kteří chtějí rozvíjet své znalosti a systematicky se zdokonalovat napříč různými obory. Výstup této práce bude mít největší přínos pro uživatele, kteří se chtějí vzdělat v oblastech programování a softwarového inženýrství, zároveň díky její rozšiřitelnosti bude možné aplikaci využít i v mnoha dalších oborech.

Kapitola 1

Analýza

V první fázi vývoje se zaměřím na analýzu. Nejprve provedu analýzu stávající aplikace na výuku angličtiny. V rámci této analýzy se zaměřím zejména na změny, které bude třeba v aplikaci provést při přechodu z domény výuky jazyků na doménu výuky softwarového inženýrství.

Dále provedu analýzu existujících aplikací, které se zaměřují na výuku programování a softwarového inženýrství. Tato analýza bude sloužit zejména jako podklad pro sepsání požadavků a pro návrh nových funkcí.

Na konci této kapitoly sepišu funkční a nefunkční požadavky, ze kterých budu dále vycházet v kapitolách návrhu a implementace.

1.1 Analýza stávající aplikace

Existující řešení je webová aplikace zaměřena na výuku anglického jazyka. Frontendová část aplikace je naprogramována pomocí technologií NextJS a React. Dále využívá knihovny, jako například Material UI. Celé uživatelské rozhraní je navrženo v nástroji Figma [1].

S backendovou částí frontend komunikuje přes REST API. Kromě aplikace existuje i kompletní mock backendového REST API, který byl vytvořen pomocí nástroje Mockoon [1].

V následujících částech textu se zaměřím nejprve na analýzu existujícího kódu a architektury a následně na jednotlivé funkcionality aplikace.

1.1.1 Analýza kódu a architektury

1.1.1.1 Design system

Aplikace využívá knihovnu Material UI, která poskytuje základní komponenty uživatelského rozhraní. Komponenty jsou přizpůsobeny především globálními styly tak, aby odpovídaly navrženému designu v nástroji Figma. Ze základ-

ních komponentů jsou následně sestaveny komplexnější komponenty, které jsou využívány v aplikaci.

V kódu je také zavedena knihovna Storybook, která umožňuje snadné prohlížení, dokumentaci a testování jednotlivých komponentů.

Celková struktura komponentů je přehledná, nicméně stojí za úvahu zdokonalit tuto strukturu vytvořením samostatného systému komponentů, který by byl oddělen od aplikace a to hlavně z následujících důvodů.

Hlavním důvodem pro vytvoření samostatné knihovny komponentů je fakt, že růstem projektu je třeba zachovat konzistenci vzhledu značky a uživatelského rozhraní napříč různými částmi projektu, kterými jsou kromě této aplikace i například webové stránky a vzdělávací a marketingové materiály. Jelikož projekt v rámci marketingu využívá videa a materiály, která jsou také vytvářena pomocí React komponentů, začíná být centrální systém komponentů nutný.

Díky vytvoření samostatného design systému by bylo možné izolovat základní komponenty a jejich styly od zbytku aplikace. To by umožnilo snadnější návrh, verzování a údržbu designu celé aplikace. Navíc by se zajistila konzistence vzhledu a funkčnosti komponentů napříč všemi částmi projektu.

1.1.1.2 Kvalita kódu

Při analýze jsem našel několik příležitostí ke zlepšení celkové kvality kódu. V průběhu let používané knihovny a nástroje prošly aktualizacemi na nové verze, které nabízejí nové funkce a možnosti. Jejich aktualizace by mohla přispět ke zvýšení kvality kódu, bezpečnosti, výkonu a celkové udržitelnosti celé aplikace.

Společně s knihovnami by bylo vhodné aktualizovat konvence a pravidla pro knihovny ESLint a Prettier a provést refaktoring kódu tak, aby odpovídal novým pravidlům.

Nakonec by bylo vhodné zvážit dockerizaci projektu, která by mohla přispět ke zjednodušení vývoje a zajištění konzistence vývojových prostředí.

1.1.1.3 Zavedení umělé inteligence pro vývoj

V průběhu posledních let došlo k výraznému rozvoji nástrojů využívajících umělou inteligenci k vývoji softwaru. Jelikož aplikace vznikala v době, kdy tyto nástroje nebyly zdaleka tak rozšířené, nejsou v aplikaci vůbec zavedeny. Proto by bylo vhodné tyto nástroje zavést do projektu tak, aby se zefektivnil celkový vývoj aplikace.

1.1.2 Analýza existujících funkcí

■ Stránka lekce

1.1.3 Návrh uživatelského rozhraní

1.2 Analýza existujících řešení

V této sekci se budu věnovat analýze existujících aplikací, které vyučují softwarové inženýrství a programování. Cílem této analýzy je zjistit, jakým způsobem existující aplikace přistupují výuce programování, jakou mají uživatelskou zkušenost a uživatelská rozhraní, jakými způsoby motivují uživatele k učení, jaké funkce poskytují zdarma a jaké jsou placené, a jaké jsou silné a slabé stránky těchto aplikací. Na konci této sekce se dále zaměřím na to, jakými způsoby se může aplikace, kterou budu v rámci této práce vytvářet, odlišit od existujících aplikací.

Pro analýzu jsem se rozhodl vybrat aplikace, které se podobají vizi tohoto projektu a budou tak přímou konkurencí vznikající aplikace. Vybíral jsem tedy mezi nejpoužívanějšími aplikacemi určenými pro samouky, které vyučují softwarové inženýrství a programování pomocí krátkých a interaktivních lekcí. Do analýzy jsem tak zahrnul aplikace Codecademy, Sololearn, Mimo a Scrimba.

Poznatky z této analýzy využiji při sepisování požadavků této aplikace, návrhu nových funkcí a nového vzhledu uživatelského rozhraní.

1.2.1 Mimo

První analyzovanou aplikací je aplikace Mimo¹, která se nejvíce blíží vizi tohoto projektu a bude tak přímou konkurencí vznikající aplikace. Mimo se zaměřuje zejména na výuku moderních programovacích jazyků a frameworků. V době psaní této práce Mimo nabízí 9 kurzů, 4 tzv. kariérní cesty a má napříč platformami přes 40 milionů registrovaných uživatelů [2]. Mimo je k dispozici jako webová aplikace a dále také jako mobilní aplikace na platformách Android a iOS. Pro analýzu použiji primárně aplikaci na platformě Android.

1.2.1.1 Vzhled a uživatelská zkušenost

Mimo se od dalších konkurentů liší zejména svým vzhledem uživatelského rozhraní a stylem výuky. Uživatelské rozhraní je velice minimalistické a přehledné. Místy může aplikace působit až příliš jednoduchým dojmem. Aplikace využívá fialovou monochromatickou paletu barev a celé rozhraní využívá spíše tmavší barvy. V rámci typografie Mimo využívá pro některé nadpisy neproporcionální písmo. To lze pozorovat zejména v mobilní aplikaci a na webových stránkách. Pro textové prvky pak využívá bezpatkové písmo Aeonik.

Rozložení prvků na obrazovce je přehledné. Na hlavní stránce kurzu jsou umístěny jednotlivé lekce, které jsou vyobrazeny ve formě tlačítek na čáře připomínající cestu kurzem. Až po kliknutí na danou lekci se zobrazí její název

¹Dostupné na: <https://mimo.org/>

a možnost spuštění dané lekce. Toto vyobrazení lekcí může na uživatele působit vizuálně poutavým dojmem, nicméně může být obtížné se pomocí něj orientovat v kurzu, protože o lekcích nejsou na první pohled zobrazeny žádné podrobnější informace.

Pro navigaci v mobilní aplikaci používá Mimo velice jednoduchou spodní navigační lištu s pouze pěti prvky. Díky tomu je navigace v aplikaci velice snadná a rychlá.

Webová verze aplikace Mimo je velice podobná mobilní verzi. Rozdíl je především v tom, že ve webové aplikaci jsou lekce vyobrazeny ve formě seznamu. Pro navigaci je použita horní navigační lišta a pro navigaci mezi kapitolami kurzu pak boční navigační lišta.

V aplikaci jsou použity animace a efekty velice zřídka. Některé prvky uživatelského rozhraní jsou animovány při jejich zobrazení. Při učení jsou také použity animace pro přechody mezi jednotlivými cvičeními.

1.2.1.2 Způsoby učení

Jednotlivé kurzy jsou uspořádány do sekcí, které obsahují jednotlivé lekce. Lekce jsou uspořádány lineárně a uživatel se je musí učit v daném pořadí. Kromě kurzů Mimo nabízí tzv. kariérní cesty, které mají podobnou strukturu jako kurzy, nicméně obsahují více témat.

Učení lze spustit kliknutím na danou lekci. Při učení se uživateli zobrazuje sekvence stránek na nichž se nachází buď vysvětlení dané látky nebo cvičení na procvičení dané látky. Při vyplňování cvičení je typicky uživateli zobrazena otázka, na kterou musí odpovědět například vyplněním textového pole v kódu. Po vyplnění odpovědi se uživateli zobrazí, zda byla odpověď správná a u cvičení se spustitelným kódem se uživateli zobrazí výstup takového kódu. V kurzu SQL dotazů uživatel může například po vyplnění odpovědi vidět tabulku s daty, které by s daným dotazem získal. Po odpovězení ve webové aplikaci má uživatel také možnost si nechat odpověď vysvětlit v chatovém okně pomocí umělé inteligence.

V aplikaci se nachází několik typů cvičení, které lze rozdělit do následujících kategorií.

Otázka s více odpověďmi Jedním z nejčastějších typů cvičení je otázka, která typicky obsahuje ukázkou kódu a výběr z několika odpovědí. Uživatel má za cíl zvolit jednu správnou odpověď.

Doplňování do kódu Dalším častým typem cvičení je doplňování částí kódu. Aplikace uživateli v otázce zobrazí ukázkou kódu, do které je třeba doplnit typicky několik slov nebo znaků. Doplňování je v některých případech usnadněno tím, že může uživatel může slova a znaky vybírat z předdefinovaného seznamu. To může značně zrychlit procházení těchto cvičení, protože uživatel nemusí vše psát ručně.

Dále aplikace ve cvičeních využívá řadu komponentů pro vysvětlení a lepší pochopení dané látky.

Tabulky V některých kurzech, zejména v kurzech vyučující témata týkající se databází, jsou využívány tabulky s daty. Ty se zobrazují jednak jako součástí zadání cvičení a jednak jako výstup po vyplnění daného cvičení.

Výstup kódu v konzoli Po vyplnění cvičení, které v zadání obsahuje spustitelný kód, může být uživateli zobrazena konzole s výstupem takového kódu. Tyto komponenty jsou typicky zobrazovány například u kurzů, které vyučují konkrétní programovací jazyky.

Zobrazení webové stránky Po vyplnění cvičení, které v zadání obsahuje kód a zaměřuje se na webové technologie, je typicky zobrazena webová stránka, která reprezentuje výstup daného kódu. Tento typ cvičení je využíván zejména u výuky jazyků, jako například HTML, CSS nebo JavaScript. Dále je tento typ cvičení také využíván pro výuku frontend frameworků.

1.2.1.3 Způsoby motivace uživatele

Mimo používá různé způsoby motivace uživatelů k učení. Jedním z těchto způsobů je gamifikace, kterou lze definovat jako použití prvků herního designu v mimoherních kontextech [3] (překlad autora). V aplikaci Mimo jsem identifikoval následující formy gamifikace.

Virtuální měna V aplikaci je možné za plnění cvičení získávat virtuální měnu, za kterou je následně možné si kupovat různé položky v obchodě aplikace.

Streak Aplikace zaznamenává, kolik dní v řadě uživatel splnil alespoň jednu lekci. Pokud uživatel daný den nesplnil žádnou lekci, je mu jeho streak resetován. V obchodě aplikace si může uživatel zakoupit několik položek, které mu pomohou zachovat streak i přes to, že se daný den zapomněl učit. Uživatel si tak může preventivně zakoupit položku, která mu zajistí, že se streak nezruší, pokud se zapomene některý den učit. Svůj streak může uživatel také zpětně napravit zakoupením příslušné položky.

Streak výzva V obchodě aplikace může uživatel za virtuální měnu zakoupit tzv. streak výzvu. Pokud uživatel splní výzvu, která spočívá v učení se každý den po dobu 7 dní, dostane jako odměnu určité množství virtuální měny.

Srdíčka Uživatel má k dispozici 5 srdíček, o která přichází, pokud neodpoví správně na danou otázku v průběhu cvičení. Srdíčka se obnovují po určitém čase a uživatel je také může zakoupit za virtuální měnu.

Body XP Za plnění jednotlivých lekcí v aplikaci uživatel může získat tzv. body XP. Tyto body jsou následně zobrazeny na jeho profilu a ovlivňují jeho pozici v lize učení.

Liga učení Uživatelé jsou každý týden rozřazeni do ligy, neboli skupiny uživatelů, na základě toho, kolik bodů XP získali v předchozím týdnu. Na profilu uživatele se pak zobrazuje, ve které lize se nachází.

Systém přátel Mimo umožňuje uživatelům si přidávat ostatní uživatele do přátel a sledovat tak jejich pokrok v aplikaci.

Ikona aplikace Mimo umožňuje uživateli si za virtuální měnu zakoupit a změnit ikonu aplikace.

1.2.1.4 Způsoby monetizace

Mimo nabízí uživatelům zdarma pouze prvních několik lekcí každého kurzu. Při učení se uživatelům zobrazují reklamy a učení je omezeno funkcí srdíček.

Uživatelé, kteří si zaplatí premium účet, se mohou všechny kurzy a kariérní cesty učit bez omezení. Při učení se jim nezobrazují reklamy a nejsou omezeni funkcí srdíček. Uživatelé s premium účtem si navíc mohou po dokončení kurzu vygenerovat a stáhnout certifikát o dokončení daného kurzu.

Aplikace nabízí sedmidenní zkušební verzi premium účtu zdarma na webových stránkách Mimo. Zkušební verzi lze také získat pozváním přátel do aplikace.

1.2.1.5 Silné a slabé stránky

Silnou stránkou aplikace Mimo je její jednoduchost používání a krátké lekce. Uživatel se tak může snadno učit v průběhu dne čistě s využitím mobilu, což může být praktické zejména pro uživatele, kteří mají omezený čas na učení.

Další silnou stránkou je použitá gamifikace, která může vést ke zvýšené motivaci uživatelů používat aplikaci pravidelně.

Možnou nevýhodou minimalistického vzhledu aplikace může být pocit, který uživatelské rozhraní svým minimalismem vyvolává. Aplikace může místy působit až příliš jednoduše a vyvolat tak dojem, že je příliš jednoduchá na to, aby vyučovala komplexní a pokročilá témata, kterým je například softwarové inženýrství. To by mohlo některé uživatele odradit, zejména ty, kteří se potřebují daná témata naučit více do hloubky.

Další potenciální nevýhodou je samotná navigace v kurzu. Lekce kurzu jsou vyobrazeny vizuálně poutavým způsobem, nicméně najít konkrétní lekci kurzu může být pro uživatele velice obtížné, zejména v mobilní verzi aplikace.

1.2.1.6 Další poznatky

Mimo kromě základních funkcí poskytuje také tzv. pískoviště, kde může uživatel vytvářet a spouštět svůj kód a vidět jeho výstup. V rámci této funkce jsou k dispozici pouze jazyky HTML, CSS a JavaScript a dále framework React.

Aplikace také poskytuje funkci, v rámci které může uživatel vytvořit vlastní webovou aplikaci přímo uvnitř Mimo. V rámci této funkce uživatel zadá umělé inteligenci, co chce vytvořit. Mimo pak uživateli nastaví virtuální prostředí se systémem souborů, kde umělá inteligence celý projekt připraví. Uživatel pak může tento projekt upravovat a spouštět přímo v Mimo. Uživatel si může v rámci této funkce zobrazit danou webovou stránku, soubory projektu a také jednoduché zobrazení tabulek v databázi. Celý projekt pak uživatel může publikovat na veřejně dostupné webové adrese. Tato funkce je pro uživatele bez premium účtu značně omezená.

1.2.2 Codecademy

Druhou analyzovanou aplikací je aplikace Codecademy². Codecademy je jednou z nejpopulárnějších platforem na výuku programování a softwarového inženýrství. Nabízí přes 800 kurzů [4] a má napříč platformami přes 50 milionů uživatelů [5]. Codecademy je k dispozici jako webová aplikace a dále také jako aplikace Codecademy Go, která je k dispozici na platformách Android a iOS.

Mobilní aplikace Codecademy Go obsahuje pouze jednoduché funkce pro opakování a procvičování látky. Hlavní náplň kurzů se pak nachází ve webové aplikaci Codecademy. Proto se v této analýze primárně zaměřím na webovou aplikaci, nicméně zahrnu do ní i poznatky ze zmíněné mobilní aplikace.

1.2.2.1 Vzhled a uživatelská zkušenost

Codecademy na první pohled působí mnohem komplexnějším dojmem než dříve analyzovaná aplikace Mimo. Design uživatelského rozhraní je spíše minimalistický, nicméně místy může stále působit příliš složitě. V aplikaci je jako barva pozadí využívána primárně světle růžová barva. Při učení je pak pro pozadí využívána bílá, černá nebo tmavě šedá barva. Pro tlačítka a další důležité prvky rozhraní pak aplikace využívá vysoce kontrastní barvy jako například modrou nebo žlutou. Co se typografie týče, Codecademy využívá napříč celou aplikací, s výjimkou bloků kódu, bezpatkové písmo Apercu.

Rozložení prvků na obrazovce je poměrně přehledné a logické, nicméně některé prvky rozhraní mohou být pro uživatele zbytečně matoucí nebo složité. Kupříkladu hned na hlavní stránce kurzu se nachází výrazné tlačítko pro smazání pokroku celého kurzu. Přestože je tato funkce velice důležitá, nejspíše nebude tak často využívána a pozice a výraznost tohoto tlačítka může akorát mást uživatele a odpoutávat jeho pozornost.

²Dostupné na: <https://www.codecademy.com/>

Na hlavní stránce kurzu je obsah kurzu rozdělen do seznamu rozbalovacích modulů, v nichž se mohou nacházet různé lekce, kvízy nebo projekty. Struktura kurzu je tak uspořádána přehledně a je snadné se v ní orientovat.

Pro navigaci v aplikaci je využita horní navigační lišta, ve které se nachází rozbalovací menu s odkazy na všechny důležité části aplikace. Dále bývá na jednotlivých stránkách využívána levá boční navigační lišta, která slouží k navigaci mezi funkcemi, které jsou relevantní pro danou stránku.

Mobilní aplikace Codecademy Go je výrazně jednodušší, co se funkcí týče. Umožňuje základní funkce zapisování si kurzů a učení se jednotlivých témat. V rámci kurzu jsou k dispozici pouze výrazně zjednodušené lekce, které se liší od lekcí a aktivit ve webové aplikaci.

V aplikaci jsou animace a efekty použity velice zřídka nebo téměř vůbec. Celá stránka tak působí velice statickým dojmem.

1.2.2.2 Způsoby učení

Co se struktury kurzů týče, Codecademy nabízí hned několik druhů kurzů. Mezi ně patří kurzy, kariérní cesty a cesty schopností. Všechny tyto druhy kurzů mají stejnou strukturu a liší se zejména svojí obsáhlostí. Dále Codecademy nabízí řadu doplňujících materiálů, kterými jsou různé nástroje pro učení, dokumentace, videa, články a další.

Jednotlivé kurzy jsou rozděleny do jednotlivých modulů. V rámci nich se nachází různé lekce, kvízy, projekty, články nebo další aktivity.

Na hlavní stránce kurzu je uživateli zobrazeno tlačítko "Pokračovat v učení", které uživateli spustí lekci nebo aktivitu, u které uživatel naposledy skončil. Jednotlivé lekce typicky skládají z cvičení, která zabírají přibližně 10 minut.

Cvičení se skládá z textového vysvětlení látky a dále jednotlivých úkolů. Uživatel má k dispozici textový editor, ve kterém může psát a editovat předpřipravený kód. Dále má k dispozici okno, ve kterém je vidět výstup daného kódu.

Kromě zmíněných prvků cvičení si může uživatel také otevřít tzv. tahák se shrnutím daného tématu. Dále si může uživatel spustit přiložené YouTube video, ve kterém učitel prochází daným cvičením a vysvětluje jednotlivé kroky.

Učení je přehledné a uživatel má k dispozici dostatek materiálů pro procvičení a pochopení dané látky. Potenciální nevýhodou může být struktura samotných cvičení. Uživatel je nucen si před každým cvičením přečíst několik stránek textu, což může být pro některé uživatele zdlouhavé a nezáživné.

V rámci učení má uživatel k dispozici AI asistenta, který mu v chatovém okně může vysvětlit danou látku nebo poradit s řešením daného úkolu.

V rámci lekcí jsou uživateli zobrazována především cvičení, v nichž uživatel doplňuje a přepisuje předpřipravený kód přímo v editoru nebo píše kód od začátku na základě zadání.

V rámci kvízů jsou uživateli zobrazovány různé otázky, typicky formou výběru z několika odpovědí nebo výběru zda je daný výrok pravdivý nebo

nepravdivý.

Jednou z aktivit jsou takzvané projekty, které svou strukturou připomínají delší cvičení v jednotlivých lekcích. Hlavním rozdílem je, že uživatel nemá možnost nahlédnout do správného řešení a jeho řešení není nijak kontrolováno. Uživatel tak musí daný úkol vyřešit sám bez pomoci. Jednotlivé úkoly pak sám může označit za splněné.

1.2.2.3 Způsoby motivace uživatele

Na rozdíl od aplikace Mimo nepoužívá Codecademy zdaleka tolik forem gamifikace. Aplikace ovšem motivuje uživatele několika následujícími způsoby.

Ukazatele pokroku Uživatel má jak na domovské stránce, tak na stránce kurzu k dispozici ukazatele pokroku, které znázorňují, kolik procent daného kurzu uživatel dokončil.

Ocenění Za splnění určitých cílů v rámci učení uživatel může získat různá ocenění, která se následně zobrazují na jeho profilu ve formě odznaků.

Certifikáty Codecademy nabízí dva typy certifikátů. První typ certifikátu uživatel může získat po dokončení určitého kurzu. Druhý typ certifikátu je tzn. profesní certifikát. Ten může uživatel získat, pokud splní všechny testy v rámci dané kariérní cesty.

Streak Podobně jako v aplikaci Mimo zaznamenává Codecademy streak uživatele, neboli počet dní, po které se uživatel alespoň jednou denně učil bez přerušení.

Sledování pokroku dovedností Unikátní funkcí Codecademy je sledování pokroku u dané dovednosti. Když se člověk například učí jazyk JavaScript, zvyšuje se mu dovednost vývoje webových aplikací. Uživatel si pak může u každé dovednosti zobrazit jeho úroveň, jak dobře danou látku ovládá a co by se měl například ještě naučit.

Celkově Codecademy obsahuje několik způsobů motivace uživatelů, nicméně zdá se, že má v aplikaci gamifikace spíše vedlejší roli.

1.2.2.4 Způsoby monetizace

Codecademy, na rozdíl od ostatních aplikací, neobsahuje žádné reklamy. Místo toho nabízí uživatelům několik různých modelů předplatného. Předplatné se dá rozdělit do kategorie pro samouky, studenty a firmy.

V rámci kategorie předplatného pro samouky Codecademy nabízí uživatelům některé základní kurzy zdarma. V rámci předplatného Plus se pak mohou

uživatelé učit neomezené množství kurzů a dostanou přístup k cestám schopností. Dále mohou neomezeně využívat AI asistenta při učení. Poslední předplatné Pro umožňuje uživatelům se učit pomocí kariérní cesty, získávat profesní certifikáty. Uživatelé s tímto předplatným mohou využívat další funkce aplikace, jako například simulátor pracovních pohovorů, programátorské výzvy a další.

Codecademy dále nabízí předplatné pro studenty. Toto předplatné je takřka stejné jako Pro předplatné pro samouky. Hlavním rozdílem je snížená cena, pokud uživatel doloží jeho status studenta.

Poslední kategorií předplatného je předplatné pro firmy. Toto předplatné má dvě varianty Teams a Enterprise. Teams nabízí podobné funkce jako Pro předplatné pro samouky s rozdílem, že firma dostane navíc možnost spravovat jednotlivé účty uživatelů, přiřazovat je do skupin a získávat přehledy o učení uživatelů. Enterprise navíc nabízí možnost integrací dalších aplikací a hodiny vedené online instruktorem.

1.2.2.5 Silné a slabé stránky

Jednou ze silných stránek Codecademy je velké množství výukových materiálů, které stránka nabízí. Uživatelé mají přístup ke kvízům, lekcím, videím, projektům, shrnutím látky a dalším nástrojům, které jim umožní se naučit danou látku.

Další silnou stránkou Codecademy je minimalistické uživatelské rozhraní. Přestože se některé části uživatelského rozhraní mohou zdát zbytečně složité, aplikace je celkově velice přehledná a snadná na používání, obzvláště pokud se vezme v úvahu množství materiálů, kurzů a funkcí, které aplikace nabízí.

Jednou z potenciálních nevýhod Codecademy je právě samotné zobrazování vzdělávacích materiálů při učení. Na dané stránce cvičení má uživatel například přístup k vysvětlení látky ve formě textu, zadání úkolu, videu vysvětlující látku, AI asistentovi, shrnutí látky, komunitní sekci a dalším podpůrným materiálům. Když uživatel prochází jednotlivými cvičeními, může se pak cítit přehlacen tím, že mu jsou všechny informace zobrazovány na jedné stránce.

Další potenciální nevýhodou Codecademy je nedostatek motivace uživatelů. Aplikace sice obsahuje několik forem gamifikace a motivace uživatele, nicméně zdá se, že tyto způsoby motivace mají v aplikaci spíše vedlejší roli.

Poslední nevýhodou této aplikace může být to, že je uživatel sám zodpovědný za opakování si dané látky. Pokud například uživatel dokončí kurz, aplikace předpokládá, že danou látku uživatel zná a nepomáhá mu si danou látku opakovat. Uživatel tak může například po několika měsících látku zapomenout a pokud by ji následně potřeboval, musel by projít kurzem znovu.

1.2.2.6 Další poznatky

Kromě výše zmíněných výhod a nevýhod jsem v rámci analýzy narazil na několik funkcí, které stojí za zmínku.

Formátování kódu v editoru V rámci cvičení má uživatel k dispozici tlačítko, kterým může automaticky zformátovat napsaný kód v editoru. To může být velice užitečné a přispívat ke kvalitní uživatelské zkušenosti, protože uživatel nemusí kód formátovat manuálně.

Studijní plán Codecademy umožňuje uživateli si vygenerovat studijní plán. V rámci plánu si může nastavit kolik dní v týdnu se chce učit a jak dlouho. Aplikace mu pak každý den doporučí konkrétní lekce a aktivity, které má daný den projít. Limitací této funkce je, že si uživatel může sestavit plán pouze pro daný kurz a nemá možnost si vygenerovat plán, který by obsahoval více kurzů zároveň. Tato funkce může být nicméně pro uživatele velice užitečná a může značně zjednodušit každodenní učení a mít tak pozitivní vliv na uživatelskou zkušenost.

1.2.3 Sololearn

Sololearn³ je jednou z nejpopulárnějších aplikací na učení se softwarového inženýrství a programování. Nabízí přes 40 kurzů na různá témata napříč softwarovým inženýrstvím a má napříč platformami přes 58 milionů registrovaných uživatelů [6].

Sololearn je k dispozici jako webová aplikace a dále také jako mobilní aplikace na platformách Android a iOS. Pro analýzu použijí mobilní verzi aplikace pro platformu Android.

1.2.3.1 Vzhled a uživatelská zkušenost

Design aplikace může na první pohled působit starším a lehce zvláštním dojmem, zejména kvůli nekonzistentnímu použití komponentů knihovny Material UI a bez dodržování doporučených standardů designu Material Design 2 [7], podle kterých se tato knihovna řídí [8]. Uživatelské rozhraní tak místy působí příliš nepřehledně. Co se palety barev týče, používá Sololearn světle modrou jako primární barvu a světle zelenou jako sekundární barvu. Pro pozadí a plochy pak používá bílou a modrošedou barvu. V rámci typografie je napříč celou aplikací použito písmo Fira Sans, jen pro bloky kódu je použito monospace písmo Fira Mono.

Rozložení prvků na obrazovce je poměrně přehledné a logické. Místy může působit nepřehledně kvůli malému prostoru mezi jednotlivými prvky, malé velikosti textu a většímu nahuštění prvků uživatelského rozhraní na jednotlivých stránkách obrazovce.

Na hlavní stránce kurzu jsou lekce seskupeny do sekcí, které může uživatel kliknutím rozbalit. Díky tomu si může jednoduše získat přehled o celém kurzu a rozbalit části, které ho zajímají.

³Dostupné na: <https://www.sololearn.com/>

Pro navigaci je v aplikaci použita dolní navigační lišta s pěti prvky a také boční navigační lišta s ostatními funkcemi.

Webová verze aplikace Sololearn obsahuje méně funkcí, než verze mobilní. V mobilní aplikaci je kromě základních funkcí také možné plnit programátorské výzvy, soutěžit ve výzvách proti ostatním uživatelům, číst články a procházet lekce vytvořené komunitou.

Sololearn využívá animace a efekty velice zřídka. Základní animace jsou použity zejména pro zobrazování prvků na obrazovce. Nejvíce animací je využito v animovaných obrázcích, které jsou zobrazovány po dokončení lekce nebo v úvodním registračním formuláři.

1.2.3.2 Způsoby učení

Kurzy jsou v aplikaci rozděleny do sekcí, ve kterých se nacházejí jednotlivé lekce a kvízy. Díky této jednoduché struktuře je velice snadné se v kurzech orientovat. Potenciální nevýhodou této jednoduché struktury může být to, že se v delších kurzech nachází příliš mnoho sekcí a uživatel, což může být pro uživatele nepřehledné.

Pro spuštění učení uživatel přejde na stránku kurzu, kde se u nadcházející lekce nachází tlačítko pro spuštění učení. Samotné učení se velice podobá učení v aplikaci Mimo. Skládá se ze série stránek vysvětlující danou látku a krátkých cvičení, při kterých si uživatel látku procvičí. Po zodpovězení otázky si uživatel může také zobrazit diskuzi a komentáře ostatních uživatelů k danému cvičení. Na rozdíl od aplikace Mimo se po dokončení cvičení nezobrazuje automaticky výstup daného kódu. Uživateli je jen zobrazeno, zda odpověděl správně nebo špatně a může si nechat jeho odpověď vysvětlit pomocí umělé inteligence. Na konci každé lekce je uživateli zobrazeno shrnutí s klíčovými informacemi, které by si měl z lekce odnést.

Sololearn nabízí v rámci učení různé druhy cvičení. Nejčastěji se jedná o doplňování slov z nabídky do kódu nebo textu. Dalším častým typem cvičení je výběr správné odpovědi z nabídky. Sololearn také nabízí cvičení, v rámci kterého může uživatel psát celý kód a zobrazovat si jeho výstupy. Celou strukturou učení se tak velice podobá analyzované aplikaci Mimo.

1.2.3.3 Způsoby motivace uživatele

Sololearn pro motivaci uživatelů využívá několik forem gamifikace. Podobně jako v aplikaci Mimo využívá streak uživatele, srdíčka při učení, body XP, ligu učení a virtuální měnu.

Kromě těchto forem gamifikace jsou uživateli také zobrazovány na různých místech v aplikaci indikátory pokroku, které zobrazují, jakou část jednotlivých kurzů uživatel dokončil.

1.2.3.4 Způsoby monetizace

Aplikace nabízí uživatelům dva typy předplatných Sololearn Pro a Sololearn Max.

Uživatelé bez předplatného mohou procházet standardní lekce ve všech kurzech. Jediným omezením je počet srdíček, které uživatelé mají. Uživatelům jsou také po dokončení lekcí zobrazovány reklamy.

Uživatelé s předplatným Pro mají přístup k speciálním lekcím v kurzech, ve kterých si mohou procvičit programátorské úlohy zaměřené na reálné problémy z praxe. Dále mají k dispozici neomezený počet srdíček, nejsou jim zobrazovány reklamy a mají přístup k dodatečným materiálům a kvízům.

Předplatné Max kromě zmíněných výhod nabízí také možnost uživateli využít umělou inteligenci, aby mu pomohla při učení. Díky ní si uživatel může nechat vysvětlit daný kus kódu, proč byla jeho odpověď špatná, a jak by ji měl opravit.

1.2.3.5 Silné a slabé stránky

Silnou stránkou této aplikace je přehlednost a jednoduchá struktura jednotlivých kurzů. Pro obsáhlejší kurzy může být tato struktura nepřehledná, nicméně pro malé a středně velké kurzy, které se v aplikaci vyskytují, může být tato struktura vhodná.

Další silnou stránkou je samotné učení. Stránky s vysvětlením látky i cvičení se snadno používají. Látka tak může být pro uživatele mnohem stravitelnější, než například u cvičení z konkurenční aplikace Codecademy.

Poslední silnou stránkou Sololearn je motivace uživatele. Aplikace využívá řadu mechanismů pro motivaci uživatele, které mohou přimět uživatele pravidelně aplikaci používat.

Jednou ze slabých stránek Sololearn je design některých stránek. Kvůli nedodržení některých standardů designu mohou stránky působit na uživatele příliš nepřehledně a zastaralým dojmem.

Druhou slabou stránkou aplikace je délka úvodního dotazníku při registraci uživatele. Dotazník obsahuje přes 10 stránek, což může být pro některé uživatele potenciálně odrazující. Založení účtu tak trvá výrazně déle, než u ostatních aplikací.

1.2.3.6 Další poznatky

Při učení aplikace na mobilu umožňuje uživateli přejít prstem po obrazovce doleva nebo doprava a tím přecházet mezi již dokončenými cvičeními v rámci dané lekce. Díky tomu se může uživatel velice jednoduše vrátit k předchozím cvičením a stránkám, které vysvětlují danou látku.

Další funkcí, která stojí za pozornost, je dostupné a snadné přepínání mezi zapsanými kurzy. Na rozdíl od ostatních aplikací, kde uživatel musí pro přepnutí mezi kurzy přejít na samostatnou stránku, může v Sololearn uživatel

jednoduše přepnout kurz kliknutím na název kurzu v horní navigační liště, čímž se rozbalí seznam zapsaných kurzů, ze kterých si může uživatel jeden vybrat a přesunout se tak na jeho hlavní stránku. Díky tomu může uživatel snadno přecházet mezi zapsanými kurzy, aniž by se musel přesouvat na samostatnou stránku se seznamem kurzů. To může být velice užitečné zejména pro uživatele, kteří se chtějí učit více kurzů zároveň.

1.2.4 Scrimba

Posledním analyzovaným řešením je webová aplikace Scrimba⁴. Tato aplikace se od ostatních odlišuje zejména svým inovativním způsobem výuky programování, který kombinuje prostředí připomínající skutečné IDE a interaktivní výuková videa. Lekce se tak odehrávají přímo v editoru kódu, ve kterém je uživateli vysvětlována látka a zadávány úkoly. Tento přístup výuky může přinést řadu výhod oproti tradičním metodám výuky a proto jsem se rozhodl tuto aplikaci do analýzy zahrnout.

Platforma Scrimba obsahuje přes 80 kurzů [9] na různá témata napříč softwarovým inženýrstvím. V době psaní tohoto textu jsou kurzy zaměřeny zejména na webové technologie a umělou inteligenci.

Na rozdíl od ostatních analyzovaných aplikací Scrimba nemá v současné době mobilní verzi aplikace. Proto pro analýzu využijí pouze webovou aplikaci Scrimba.

1.2.4.1 Vzhled a uživatelská zkušenost

Na první pohled působí Scrimba jako přehledná a minimalistická aplikace. Ve srovnání s konkurenční aplikací Mimo používá aplikace menší velikost písma a zobrazuje na stránkách více textu. Aplikace tak může působit komplexnějším dojmem. Co se palety barev týče, používá Scrimba monochromatickou paletu modré barvy. Odstíny modré jsou použity pro pozadí, primární akce, ikony a textové prvky. Pro upozornění a některé štítky aplikace výjimečně využívá i žlutou nebo zelenou barvu. Napříč aplikací je používáno systémové písmo, tedy písmo, které je nastaveno v závislosti na operačním systému uživatele. Na systému Windows je tak použito například písmo Segoe, na zařízeních macOS písmo San Francisco a na zařízeních Android písmo Roboto [10].

Prvky jsou na obrazovce rozloženy velice přehledně a logicky. Ve srovnání s ostatními aplikacemi Scrimba používá méně prostoru mezi jednotlivými prvky uživatelského rozhraní. V důsledku toho může aplikace místy působit komplexnějším dojmem.

Na hlavní stránce kurzu jsou jednotlivé sekce, moduly a lekce vizuálně uspořádány do stromové struktury, kterou lze postupně rozbalovat. Díky tomu může uživatel jednoduše získat přehled o celém kurzu a rozbalit části, které ho zajímají. Lekce jsou tak uspořádány velice přehledně.

⁴Dostupné na: <https://scrimba.com/>

Na rozdíl od ostatních analyzovaných aplikací Scrimba nevyužívá pro navigaci horní navigační lištu, ani žádnou patičku stránky. Pro navigaci je využívána výhradně boční navigační lišta, která je umístěna na levé straně obrazovky.

Ve srovnání s ostatními aplikacemi Scrimba výrazně více využívá animace jednotlivých prvků uživatelského rozhraní a animované přechody mezi stránkami. Na většině stránek je při jejich načtení některý z prvků animován. Další výraznou animaci lze pozorovat při přejetí kurzorem nad tlačítko. Při animaci je tlačítko zvýrazněno čarou, která ho postupně obklopí po jeho obvodu.

Animace jsou napříč aplikací používány často, nicméně minimalisticky a v přiměřeném množství. Díky tomu Scrimba může působit výrazně interaktivnějším a vizuálně poutavějším dojmem, než její konkurenti.

1.2.4.2 Způsoby učení

Scrimba nabízí, podobně jako ostatní analyzované aplikace, kurzy a kariérní cesty. Kariérní cesty jsou takřka stejné jako kurzy, liší se pouze v tom, že obsahují více témat. Lekce v kurzech jsou uspořádány chronologicky a seskupeny do kapitol a podkapitol.

Unikátní funkcí této aplikace je samotné učení. Při spuštění učení je uživateli v prohlížeči zobrazen editor připomínající skutečné IDE, ve kterém je možné prohlížet soubory a editovat kód. V editoru zároveň uživatel vidí okno s výstupem daného kódu. Dále je uživateli zobrazena prezentace se slidy doplňující výklad dané látky. Všechna tato okna může uživatel v průběhu výkladu schovávat, zobrazovat a procházet dle libosti.

Při učení uživatel poslouchá audio s vysvětlením dané látky. V průběhu toho je mu buď zobrazena prezentace nebo editor, ve kterém může sledovat, jak lektor přepisuje kód a jak pohybuje svým kurzorem myši. Při výkladu může uživatel procházet libovolné soubory a přepisovat jejich obsah. Dále si může kdykoliv zobrazit okno s prezentací nebo výstupem kódu. Audio s výkladem látky může uživatel kdykoliv přetáčet zpět a nebo dopředu. V průběhu lekce jsou uživateli zadávány úkoly, které musí v editoru splnit. Zadání je provedeno lektorem v audio nahrávce. Po zadání úkolu je audio pozastaveno, dokud uživatel nesplní daný úkol.

Na rozdíl od ostatních analyzovaných aplikací Scrimba nepoužívá předdefinované typy cvičení. Veškerý výklad látky i cvičení se odehrávají přímo v editoru. Cvičení typicky spočívají v úpravě existujícího kódu nebo napsání nového kódu. Celkově je učení velice interaktivní a připomíná kombinaci videí s výkladem a skutečného IDE. Tato forma učení může být pro uživatele velice přínosná, zejména kvůli názornějšímu vysvětlení látky a interaktivnímu způsobu učení.

1.2.4.3 Způsoby motivace uživatele

Na rozdíl od ostatních aplikací Scrimba neobsahuje téměř žádné formy gamifikace. Uživatelé nesbírají žádné body, ani ocenění a nemohou spolu v aplikaci interagovat. Motivující pro uživatele ovšem může být, že aplikace jasně ukazuje, kolik procent kurzu uživatel dokončil a kolik cvičení uživatel splnil. Všechny splněné lekce jsou také označeny výraznou zelenou barvou a ikonou zaškrtnutí. Tato vizuální indikace pokroku může pro některé uživatele sloužit jako zdroj motivace.

1.2.4.4 Způsoby monetizace

Scrimba narozdíl od ostatních aplikací neobsahuje žádné reklamy a nabízí pouze jeden model předplatného.

Uživatelé bez předplatného mají k dispozici většinu kurzů zdarma. Některé kurzy a kariérní cesty jsou ovšem k dispozici pouze uživatelům s aktivním předplatným. V nich mají uživatelé bez předplatného možnost si vyzkoušet několik prvních lekcí.

Uživatelé s aktivním předplatným mají kromě základních kurzů přístup k pokročilejším kurzům a kariérním cestám. Na konci každého kurzu si navíc mohou vygenerovat a stáhnout certifikát o dokončení daného kurzu.

1.2.4.5 Silné a slabé stránky

Silnou stránkou této aplikace je její unikátní přístup k učení. Uživatelé se učí přímo v editoru připomínající IDE a veškerá látka je vysvětlena lektorem a doplněna o prezentaci s obrázky, GIFy a diagramy. Učení je tak zábavné, interaktivní a může být pro uživatele více srozumitelné, zejména u složitější látky.

Další silnou stránkou aplikace je minimalistický design a rozložení obsahu. Aplikace se velice snadno používá a je velice jednoduché najít, co uživatel potřebuje. Důvodem může být také to, že aplikace obsahuje pouze funkce, které jsou klíčové pro výuku programování a nezahluje uživatele dodatečnými materiály a nástroji.

Mírnou nevýhodou inovativního způsobu učení je samotná volnost uživatele při učení. Uživatel může v průběhu výkladu svým počínáním například omylem vypnout okno s prezentací nebo výstupem kódu. Proto se může velice jednoduše stát, že lektor v audio nahrávce mluví například o diagramu v prezentaci, který se uživateli na obrazovce nezobrazuje, protože dříve kliknul na editor, což zamezilo zobrazení okna s prezentací. Pro uživatele tak může být místy velice matoucí, kde se zrovna lektor nachází v průběhu výkladu a může být těžké výklad sledovat.

Další potenciální nevýhodou může být nedostatek motivace uživatele. Aplikace neobsahuje téměř žádné funkce, které by se zaměřovaly na dlouhodobou

motivaci uživatele. Proto může být pro uživatele obtížné se dlouhodobě přimět k používání této aplikace.

Poslední slabou stránkou aplikace Scrimba je místy nefunkční editor kódu. Protože do kódu může zároveň zapisovat virtuální lektor a uživatel, několikrát se při průchodu aplikací stalo, že byl například kurzor posunutý o několik znaků vedle, nebo že psaní do kódu nefungovalo a přepisovaly se jiné znaky, než by uživatel očekával. Tento problém může být pro uživatele velice odrazující při používání aplikace, protože se aplikace jeví jako nefunkční.

1.2.4.6 Další poznatky

V rámci analýzy jsem došel k pozoruhodnému poznatku. Na rozdíl od ostatních aplikací Scrimba nemá dedikovanou domovskou stránku, která by například uživateli prezentovala výhody aplikace a jiné marketingové materiály. Místo toho je uživateli na hlavní stránce zobrazen seznam všech kurzů a krátké video popisující, jak se aplikace používá. Uživatelské rozhraní nepřihlášeného a přihlášeného uživatele se tak téměř neliší. Uživatelé mohou prohlížet všechny kurzy bez přihlášení a mohou spustit až dvě libovolné lekce, než po nich bude aplikace vyžadovat přihlášení. Díky tomu mohou uživatelé zjistit, zda jim vyhovuje formát výuky, než se do aplikace zaregistrují.

1.2.5 Závěr analýzy

V této části textu shrnu nejdůležitější výstupy provedené analýzy konkurenčních řešení. Tyto výstupy budou sloužit jako podklad pro sestavení požadavků tohoto projektu.

1.2.5.1 Vzhled a uživatelská zkušenost

Většina analyzovaných aplikací se snaží poskytnout uživateli přehledný a snadno použitelný způsob učení. Vzhled uživatelského rozhraní je u většiny aplikací minimalistický. Tomu odpovídá i struktura stránek a samotných kurzů, která bývá velice jednoduchá.

Aplikace často používají animace a další vizuální efekty, aby udělaly učení a používání aplikace vizuálně poutavější. To může potenciálně pomoci udržet pozornost uživatele při učení. Současné řešení animace a efekty téměř nepoužívá a proto stojí za zvážení jejich použití.

1.2.5.2 Způsob výuky

Analyzované aplikace volí různé způsoby výuky. Aplikace Mimo a Sololearn učí uživatele pomocí krátkých cvičení, Codecademy pomocí delších úkolů a Scrimba pomocí vysoce interaktivních lekcí. Každý způsob výuky poskytuje určité výhody a nevýhody a bude třeba zvážit, který způsob je pro uživatele nejvhodnější.

Při učení všechny aplikace poskytují určitou formu nápovědy pomocí umělé inteligence. Ta například může uživateli vysvětlit danou látku, nebo chybu, kterou ve cvičení udělal. Tato funkce může být pro učení a uživatelskou zkušenost klíčová a bude třeba ji do aplikace zahrnout.

V rámci učení jsou uživateli zobrazovány různé typy cvičení. Typicky se jedná o doplňování slov do kódu, psaní kódu do editoru a výběr správné odpovědi z nabídky.

Ve většině aplikací je uživateli látka vysvětlena pomocí krátkých textových popisů. Výjimkou je aplikace Scrimba, ve které je látka uživateli vysvětlena pomocí audio nahrávky v kombinaci s prezentací a interaktivním editorem. Použití audio nahrávek a interaktivních prvků může být pro uživatele potenciálně poutavější a srozumitelnější, než pouhé čtení textu. Podobnou formu výuky bude vhodné zvážit při návrhu struktury učení v této práci.

Většina aplikací také zobrazuje lekce kurzu ve formě seznamu. To se zásadně liší od současné aplikace, ve které jsou lekce vizuálně uspořádány do tvaru připomínající cestu kurzem. Přestože tato struktura může být vizuálně poutavější, a například aplikace Mimo ji také využívá, bude třeba zvážit, zda-li by pro uživatele nebylo přehlednější uspořádání lekcí do přehlednějšího seznamu.

1.2.5.3 Motivace uživatelů

Ve většině aplikací jsou používány určité funkce, které mají za cíl motivovat uživatele, aby se učil a dlouhodobě používal danou aplikaci.

Mezi tyto prvky se řadí zejména různé způsoby gamifikace. Mezi nejpoužívanější způsoby v analyzovaných aplikacích patří například streak, body XP, ligy učení, srdíčka při učení a virtuální měna.

Kromě těchto prvků aplikace také často využívají indikátory pokroku v rámci daných kurzů. Dokončené lekce jsou například označovány zelenou barvou a uživateli je zobrazován procentuální pokrok v daném kurzu. Tyto ukazatele pokroku ve stávající aplikaci nejsou vůbec zahrnuty a stojí za zvážení jejich použití.

Posledním výrazným prvkem pro motivaci uživatelů jsou certifikáty o dokončení daného kurzu. Po dokončení kurzu a splnění podmínek si může uživatel ve většině aplikací stáhnout certifikát o jeho dokončení ve formátu PDF. Tento certifikát pak může uživatel například využít při hledání práce nebo prezentaci svých znalostí. Tuto funkci bude opět vhodné zvážit při sestavování požadavků tohoto projektu.

1.2.5.4 Odlišení aplikace od existujících řešení

Na závěr analýzy se zaměřím na specifické funkce, kterými se bude moci tato aplikace odlišit od analyzovaných konkurenčních řešení. Tyto funkce budou moci aplikaci poskytnout na trhu konkurenční výhodu a uživatelům lepší uživatelskou zkušenost.

Interaktivní lekce s audio výkladem Hlavní konkurenční výhodou této aplikace budou interaktivní lekce, které kombinují výhody přístupů analyzovaných aplikací. Lekce budou krátké a budou se skládat z interaktivních stránek výkladu a cvičení. Způsob výuky tak bude podobný způsobu výuky aplikací Mimo a Sololearn. Průběh učení ovšem bude doplněn o audio nahrávku s výkladem, podobně jako tomu je v aplikaci Scrimba. Uživatelům tak bude látka prezentována po malých krocích, což jim umožní látku snadněji pochopit. Aplikaci to dále umožní přesně identifikovat, se kterými koncepty má daný uživatel problém a tyto koncepty mu bude moci častěji ukazovat při opakování. Audionahrávky doplněné o animace, kód a obrázky navíc budou moci uživatelům přinést lepší uživatelskou zkušenost. Uživatelé tak nebudou muset číst dlouhé odstavce textu, ale veškerá látka jim bude tímto způsobem odprezentována a názorně vysvětlena.

Důraz na opakování látky Další konkurenční výhodou oproti analyzovaným aplikacím bude větší důraz na opakování již naučené látky. Většina konkurenčních aplikací na opakování klade velmi malý důraz. Pro uživatele tak může být obtížné si koncepty a informace dlouhodobě zapamatovat.

Stávající aplikace již obsahuje určité formy opakování pomocí metody spaced repetition [11]. Tuto formu opakování bude třeba změnit a přizpůsobit tak, aby bylo možné ji využít pro výuku softwarového inženýrství. Při návrhu aplikace tak bude třeba zvážit, jakým způsobem bude opakování látky v aplikaci prováděno.

Zaměření na gamifikaci Některé konkurenční aplikace využívají prvky gamifikace pro zvýšení a udržení motivace uživatele. Tyto prvky nejsou ovšem zdaleka použity tak efektivně, jak by mohly. Některé aplikace tyto prvky využívají velice zřídka a jiné je zase nevyužívají způsobem, jaký by byl pro uživatele optimální.

Aplikace se tak může na tento aspekt učení zaměřit a poskytnout uživatelům efektivnější a propracovanější použití gamifikace.

Zaměření na uživatelskou zkušenost Některé aplikace, jako například Sololearn nebo Scrimba, mají řadu nedostatků, co se návrhu vzhledu aplikace a uživatelské zkušenosti týče. Tyto nedostatky mohou být pro uživatele odrazující. Při vývoji této aplikace tak bude možné se více zaměřit na vzhled, uživatelskou zkušenost a testování, díky čemuž by aplikace mohla poskytnout kvalitnější uživatelskou zkušenost.

Kvalitní učení Efektivní a kvalitní učení je klíčovým aspektem vzdělávacích aplikací. Přesto konkurenční aplikace obsahují řadu nedostatků, které mohou uživateli znepříjemnit učení. Při zapnutí aplikace Sololearn je například uživatel odkázán na svůj profil, kde je zahlcen řadou nepodstatných informací.

Místo toho by například mohl být odkázán přímo na stránku kurzu, kde by mohl pokračovat v učení.

V průběhu učení je například v aplikaci Codecademy zobrazeno uživateli značné množství materiálů a informací, které pro něj mohou být velice rozptylující a snižovat tak efektivitu samotného učení.

Aplikace by se tak mohla zaměřit na předejití těchto nedostatků a poskytnout uživateli příjemnější a kvalitnější výuku.

Studijní plány V rámci učení se jednoduše může stát, že se uživatel potřebuje učit několik kurzů zároveň. Začínající programátor se například musí naučit řadu technologií a jazyků, se kterými se musí naučit pracovat.

Pokud se ovšem uživatel chce učit více kurzů zároveň, musí mezi jednotlivými kurzy neustále složité přepínat a sám plánovat, jakou látku se bude učit dál.

Aplikace by tak mohla poskytnout uživateli možnost si zapsat jednotlivé kurzy a zvolit si úroveň, které chce uživatel v kurzech dosáhnout. Následně by mu mohla předkládat lekce k naučení podle automaticky vygenerovaného studijního plánu. Uživatel by si tak mohl zapsat kurzy a následně se jen učit lekce, které mu aplikace doporučí.

Sociální aspekty Další slabou stránkou aplikací je nedostatek sociálních interakcí mezi uživateli. Aplikace často neobsahují uživatelské profily, sledování aktivit ostatních uživatelů, vyhledávání jejich profilů a sdílení pokroku mezi ostatními uživateli. Tento sociální aspekt používání může být zásadním zdrojem motivace pro některé uživatele a mohl by přispět ke zlepšení uživatelské zkušenosti.

Využití animací, přechodů a efektů Konkurenční aplikace zřídka využívají animace, přechody a efekty mezi stránkami. Přestože je třeba používat tyto prvky v přiměřeném množství, může jejich použití docílit toho, že bude aplikace pro uživatele poutavější a bude se tím moci odlišit od konkurentů.

1.3 Analýza požadavků

V této sekci sepišu funkční a nefunkční požadavky vznikající aplikace. Při sepišování požadavků budu vycházet zejména z analýzy stávajícího řešení a analýzy existujících aplikací. Dále budu také vycházet z výsledků uživatelských testů, které byly provedeny v rámci mé bakalářské práce [1] zabývající se stávající aplikací, a také z návrhu úprav do budoucna, které byly popsány ve zmíněné práci.

Po sepsání jednotlivých požadavků se zaměřím na určení jejich priorit pomocí metody MoSCoW [12]. Tato metoda umožňuje jednoduše rozdělit požadavky do čtyř kategorií dle jejich priority. Požadavky v kategorii *must have*

musí být při implementaci splněny, požadavky kategorie *should have* by měly být splněny, požadavky kategorie *could have* mohou být splněny, a požadavky kategorie *won't have* nebudou v rámci této práce implementovány.

Na základě předchozí analýzy stávajícího řešení a analýzy konkurenčních aplikací jsem identifikoval a sepsal následující funkční a nefunkční požadavky.

1.3.1 Funkční požadavky

Při analýze jsem identifikoval a sepsal následující funkční požadavky, kterým se budu věnovat v rámci této práce.

1.3.1.1 FP-1 Úvodní stránka aplikace

Priorita: Should have

Aplikace by měla mít úvodní stránku, která bude zobrazena, když uživatel otevře aplikaci a není přihlášen. Tato stránka by měla upoutat pozornost uživatele a umožnit mu vybrat si, jestli se chce přihlásit nebo registrovat.

Tento požadavek vychází z analýzy konkurence, kde většina aplikací má kromě stánky pro přihlášení a registraci i úvodní stránku s poutavou grafikou či textem, který prezentuje hlavní přínosy aplikace pro uživatele.

1.3.1.2 FP-2 Nový úvodní dotazník a doporučování kurzů

Priorita: Should have

Aplikace by měla obsahovat úvodní dotazník zaměřený na získání informací o uživateli. Tyto informace by se měly týkat zejména cílů uživatele a jeho zkušeností s programováním. Na základě dotazníku by měla aplikace nabídnout uživateli několik kurzů, které odpovídají jeho cílům a zkušenostem.

Tato funkce je inspirována konkurenční aplikací Codecademy, která využívá úvodní dotazník pro doporučování kurzů uživateli.

1.3.1.3 FP-3 Prohlížení a doporučování kurzů

Priorita: Must have

Uživatel by měl mít v aplikaci možnost zobrazit přehled dostupných kurzů a jednoduše je prohledávat. U každého kurzu by měl mít možnost se přesunout na stránku daného kurzu.

Součástí prohlížení kurzů by měla být sekce s doporučenými kurzy. Tato doporučení by měla být založena jednak na úvodním dotazníku uživatele, který vyplnil při registraci. Uživatel by měl mít možnost dotazník kdykoliv vyplnit znovu a přizpůsobit si tak svoje preference při učení.

1.3.1.4 FP-4 Stránka kurzu

Priorita: Must have

Každý kurz v aplikaci by měl mít vlastní stránku s jeho popisem a přehledem lekcí, které jsou v kurzu obsaženy. Uživatel by měl mít možnost se do kurzu přihlásit nebo se z něj odhlásit. Dále by měl mít možnost u kurzu zvolit, zda se ho chce momentálně učit nebo ne. Na základě této volby by aplikace měla uživateli zobrazovat cvičení při učení.

Tento požadavek vychází z analýzy konkurence, kde ve většině aplikací mají kurzy vlastní stránku se svým popisem a přehledem lekcí.

1.3.1.5 FP-5 Správa zapsaných kurzů

Priorita: Must have

Uživatel by měl mít možnost si zobrazit přehled zapsaných kurzů. U každého kurzu by měl uživatel vidět jeho pokrok a úroveň v rámci daného kurzu.

Uživatel by měl mít také možnost se z přehledu kurzů přesunout na stránku daného kurzu.

1.3.1.6 FP-6 Nastavení cílové úrovně kurzu

Priorita: Must have

Aplikace by měla umožnit uživateli si nastavit cílovou úroveň kurzu, tedy úroveň, které chce v uživateli daném kurzu dosáhnout. Po dosažení této úrovně by měl být kurz přepnut do režimu, v němž se uživateli bude při učení zobrazovat pouze látka k zopakování a žádné nové lekce. Uživatel si tak bude moci u každého kurzu nastavit, jaké úrovně chce dosáhnout, což mu může usnadnit učení, pokud má v aplikaci zapsaných více kurzů.

Možnost nastavení cílové úrovně by měla být k dispozici jednak při zápisu kurzu a jednak na stránce nastavení kurzu, kterou bude třeba v rámci tohoto požadavku také implementovat.

Pokud uživatel dosáhne cílové úrovně daného kurzu, mělo by se mu zobrazit oznámení, že v daném kurzu dosáhl svého cíle a že bude kurz přepnut do režimu, v němž si uživatel bude pouze opakovat danou látku. Dále by se mu měla zobrazit informace, že režim kurzu může kdykoliv změnit v nastavení.

1.3.1.7 FP-7 Kariérní cesty

Priorita: Must have

Aplikace by kromě kurzů měla nabízet tzv. kariérní cesty, které sdružují jednotlivé kurzy do skupin podle určitého tématu. Každá kariérní cesta reprezentuje určitou profesi. Každá kariérní cesta by měla mít vlastní stránku s popisem a přehledem kurzů, které jsou součástí dané kariérní cesty.

Uživatel by měl mít možnost si kariérní cestu zapsat a také mít možnost se od ní odhlásit.

Dále by měl mít uživatel možnosti si zvolit, zda se chce danou cestu momentálně učit nebo ne. Na základě této volby by aplikace měla uživateli zobrazovat cvičení při učení.

Tento požadavek vychází z analýzy konkurence, kde většina analyzovaných aplikací nabízí určitou formu kariérních cest, díky kterým se uživatel může učit témata týkající se konkrétní profese.

1.3.1.8 FP-8 Stránka s lekcemi kurzu

Priorita: Must have

Ve stávající aplikaci jsou na stránce s lekcemi kurzu lekce vyobrazeny jako body na čáře připomínající cestu, po které se uživatel při postupování kurzem pohybuje. Stránka tak připomíná vizuálně mapu s cestou, po které se uživatel pohybuje.

Tato mapa sice může zachytit svým vzhledem pozornost uživatele, nicméně má několik zásadních nevýhod z hlediska použitelnosti. Lekce jsou na mapě reprezentovány pouze ikonou a zkráceným názvem. Kvůli tomu může být pro uživatele obtížné na první pohled poznat, co je obsahem dané lekce. Dále, pokud se uživatel snaží najít konkrétní lekci v aplikaci, může to pro něj být velice obtížné, pokud mapa učení obsahuje větší množství lekcí.

Z těchto důvodů je třeba upravit stránku s lekcemi kurzu tak, aby byla přehlednější a umožňovala uživateli se lépe v lekcích orientovat.

1.3.1.9 FP-9 Studijní plán

Priorita: Must have

Aplikace by měla obsahovat stránku studijního plánu uživatele. Na této stránce by měl mít uživatel možnost si zapsaných kurzů zvolit, které se chce učit a jaké úrovně v nich chce dosáhnout. Podle studijního plánu pak bude aplikace doporučovat uživateli lekce při učení.

U každého kurzu bude uživatel moci přepínat mezi následujícími režimy. Tím bude moci zvolit, které zapsané kurzy se chce učit a které ne.

- **Zapnutý** – uživateli bude aplikace zobrazovat z daného kurzu novou látku k učení i látku k zopakování
- **Pouze opakování** – uživateli bude aplikace zobrazovat pouze látku k zopakování z daného kurzu
- **Vypnutý** – aplikace nebude uživateli zobrazovat žádnou látku z daného kurzu

Dále bude moci u každého kurzu nastavit úroveň, které chce v kurzu dosáhnout.

Funkce studijního plánu vychází z analýzy konkurenční aplikace Codecademy, která umožňuje uživateli si vygenerovat studijní plán. Funkce dále vychází z problému, který nastává v konkurenčních aplikacích, pokud se uživatel chce učit více kurzů zároveň. Uživatel tak bývá nucen přepínat manuálně mezi jednotlivými kurzy a sám si plánovat, kterou látku se bude učit. Studijní plán

by měl tomuto problému předejít a umožnit uživateli se jednoduše učit látku napříč mnoha kurzy.

1.3.1.10 FP-10 Učení se podle studijního plánu

Priorita: Must have

Aplikace by měla uživateli poskytnout možnost se jednoduše přesunout na stránku učení. Na této stránce by měla aplikace uživateli zobrazit doporučenou lekci nebo látku k zopakování, které budou vybrány na základě znalostí uživatele a jeho studijního plánu. Lekce a látka k zopakování bude vybírána napříč všemi relevantními kurzy dle studijního plánu.

Stránka učení by zároveň měla být nastavena jako výchozí stránka, na kterou bude uživatel přesměrován po spuštění aplikace.

Tato funkce má dva hlavní cíle. Prvním cílem je umožnit uživateli jednoduše spustit učení z libovolného místa v aplikaci a zamezit tak složitému přepínání mezi kurzy, které může nastávat v konkurenčních aplikacích, když se uživatel chce učit více kurzů zároveň.

Druhým cílem této funkce je využít princip návrhu uživatelského rozhraní *microbreaks* [13]. Hlavní myšlenkou tohoto principu je přizpůsobit aplikaci tak, aby ji mohl uživatel zapnout a použít kdykoliv, kdy má v průběhu dne pár minut volného času. Když uživatel například čeká chvíli na zastávce, může aplikaci zapnout a využít tak efektivně svůj čas. Klíčovým prvkem tohoto principu je umožnit uživateli se ihned po spuštění aplikace přesunout k dané aktivitě, tedy v případě této aplikace, k samotnému učení. Tento princip je využíván například aplikacemi sociálních sítí, které jsou přizpůsobeny na to, aby je mohl uživatel zapnout kdykoliv, kdy má chvilku volného času.

1.3.1.11 FP-11 Cvičení a komponenty pro výuku softwarového inženýrství

Priorita: Must have

Aplikace se bude zaměřovat na výuku základů softwarového inženýrství, a proto bude obsahovat cvičení na různá témata z tohoto oboru. Konkrétně by aplikace měla obsahovat následující cvičení.

- Otázka s výběrem správné odpovědi
- Otázka a kód s vynechanými políčky, do kterých uživatel vyplní slova nebo znaky z nabídky
- Otázka a kód s vynechanými políčky, do kterých uživatel sám vyplní znak nebo slovo

Ve cvičeních, v nichž uživatel vyplňuje komplexnější kód, by mu měla aplikace zobrazit po vyplnění odpovědi výstup kódu. Ten může být zobrazen v následujících formátech.

- Okno s jednoduchou HTML stránkou, která může obsahovat CSS styly a JavaScript kód.
- Okno s textem vizuálně připomínajícím výstup v terminálu.
- Okno s tabulkou vyplněnou daty.

Při učení bude dále možné zobrazovat uživateli následující komponenty.

- Krátká ukázka kódu na jednom řádku textu
- Jednoduchý editor kódu s možností zobrazení více záložek reprezentující různé soubory nebo okna s výstupem kódu
- Obrázky ve formátech PNG, JPG, SVG a GIF
- Odstavce textu
- Nadpisy
- Tabulky
- Seznamy číslované a nečíslované

U komponent, v nichž uživatel může psát celý kód, by měla aplikace umožnit uživateli jednoduše zformátovat daný kód. Tato funkce vychází z analýzy konkurence Codecademy, která tuto funkci nabízí a může přispět k lepší uživatelské zkušenosti.

1.3.1.12 FP-12 Nová struktura učení

Priorita: Must have

Učení by mělo být strukturováno tak, aby uživateli byla jednak po malých krocích vysvětlena probíraná látka a jednak aby si uživatel mohl danou látku procvičit. Při procvičování látky by měla aplikace sbírat data o tom, jak uživatel danému tématu rozumí. Tento způsob výuky je inspirován analyzovanými aplikacemi Mimo a Duolingo, které podobnou formu výuky využívají.

Při učení by aplikace měla uživateli vysvětlovat látku a zadávat cvičení pomocí audio nahrávky, která bude doplněna o další komponenty, které pomohou uživateli lépe porozumět dané látce. Díky tomu bude aplikace kombinovat přístup konkurenční aplikace Mimo, která využívá různé komponenty pro vysvětlení látky, a přístup konkurenční aplikace Scrimba, v níž je látka vyučována pomocí interaktivního prostředí s audio nahrávkou vysvětlující danou látku.

Aplikace by měla brát v potaz situaci, kdy uživatel nebude moci poslouchat audio a měla by uživateli umožnit se učit i bez něj.

Tato struktura učení se zásadně liší od struktury učení v původní aplikaci, kde probíraná látka nebyla uživateli při učení vysvětlena a učení nebylo doplněno o audio nahrávku vysvětlující danou látku.

Při řešení tohoto požadavku je třeba také brát v potaz výsledky uživatelského testování stávajícího řešení, které bylo provedené v rámci mé bakalářské práce [1]. Výsledky tohoto testování ukázaly, že aplikace nedává uživateli dostatečně jasnou zpětnou vazbu o tom, zda-li odpověděl při plnění cvičení správně nebo špatně. To bude třeba zohlednit při návrhu a implementaci nového způsobu učení.

1.3.1.13 FP-13 Chatbot

Priorita: Should have

Aplikace by měla při učení poskytovat premium uživateli chatbota, který mu bude moci vysvětlit, proč byla jeho odpověď ve cvičení špatná nebo mu podrobněji vysvětlit, co daný kód ve cvičení znamená a jak funguje.

1.3.1.14 FP-14 Nahlašování problému při učení

Priorita: Should have

Uživatel by měl mít při učení možnost u každého cvičení nebo vysvětlení látky nahlásit problém, který při dané aktivitě nastal. Při nahlašování by měl mít možnost zvolit o který typ problému z následujícího výběru se jedná.

- Uživatel považuje svou odpověď ve cvičení za správnou, i přes to, že ji aplikace označila jako špatnou.
- Uživatel považuje svou odpověď ve cvičení za špatnou, i přes to, že ji aplikace označila jako správnou.
- Zadání cvičení je pro uživatele matoucí nebo nejasné.
- Vysvětlení látky je pro uživatele matoucí nebo nejasné.
- Nastala jiná chyba. V tomto případě by měl uživatel mít možnost sám uvést, o jakou chybu se jedná.

Uživatelé s rolí korektora by měli kromě předchozích možností nahlášení problému, mít také k dispozici následující možnosti.

- Vysvětlení látky je příliš dlouhé nebo příliš krátké.
- Cvičení je příliš těžké nebo příliš snadné.
- Cvičení nedostatečně testuje probíraný koncept.
- Struktura lekce by měla být upravena. Tato možnost se využije například pokud v lekci není dostatek cvičení nebo vysvětlení látky.

Korektoři by také měli mít možnost u každé odpovědi detailněji popsat vlastními slovy daný problém.

Nahlášení problému od uživatelů s rolí korektora by mělo být označeno tak, aby bylo možné jej rozlišit od nahlášení problému běžných uživatelů. Role korektora bude nastavována uživatelům mimo frontend této aplikace. Tuto roli využijí zejména lidé zodpovědní za tvorbu a korekci jednotlivých kurzů.

1.3.1.15 FP-15 Statistiky o učení

Priorita: Could have

Aplikace by měla při učení uživatele nově sbírat data o tom, jak dlouho uživatel prováděl jednotlivá cvičení nebo procházel vysvětlení látky. Tato data by měla být využita k tomu, aby se na konci učení uživateli zobrazila informace, jak dlouho se učil.

Dále by měla být tato data odeslána do backendové části aplikace, kde budou následně zpracována a použita pro úpravu odhadů časové náročnosti jednotlivých cvičení. Touto funkcí backendové části aplikace se zabývá kolega Bc. Jakub Vondráček v jeho diplomové práci.

1.3.1.16 FP-16 Stav energie

Priorita: Should have

Aplikace by měla uživateli zobrazovat jeho stav energie, který je reprezentovaný body energie. Učením se budou body energie snižovat. Pokud uživatel všechny body energie spotřebuje, nebude mu umožněno se dále učit. Body energie se budou v průběhu času obnovovat. Uživatelé s premium účtem budou mít neomezené množství energie, tedy učení nebude body energie snižovat.

Tento požadavek je inspirován analyzovanou aplikací Duolingo, která podobnou funkci nabízí. Tato funkce slouží primárně k motivaci uživatele, aby se učil pravidelně každý den a dále také k tomu, aby byl uživatel motivován si zakoupit premium účet.

1.3.1.17 FP-17 Registrace a přihlášení pomocí Google a Apple

Priorita: Should have

Uživatel by měl mít možnost se registrovat a přihlásit do aplikace pomocí účtu Google nebo Apple.

Tento požadavek vychází z analýzy konkurence, kde většina konkurenčních aplikací tuto možnost nabízí.

1.3.1.18 FP-18 Zpřehlednění navigace v aplikaci

Priorita: Should have

Aplikace by měla poskytovat uživateli přehlednější a jednodušší způsob navigace mezi jednotlivými stránkami. Z výsledků uživatelského testování v mé

bakalářské práci [1] zabývajících se stávající aplikací vyplynulo, že mají uživatelé problém s navigací mezi jednotlivými stránkami aplikace, zejména mezi jednotlivými úrovněmi mapy učení v daném kurzu. Dále, také z důvodu přidání nových stránek do aplikace, bude třeba zajistit, aby navigační prvky v aplikaci byly pro uživatele přehledné a snadné na používání.

Změnu navigace bude třeba provést zejména v hlavní navigační liště, bočním navigačním panelu a stránkou sloužící k přepínání mezi jednotlivými kurzy.

Cílem této změny je zpřehlednit aplikaci a zjednodušit uživateli navigaci mezi jednotlivými stránkami.

1.3.1.19 FP-19 Tmavý režim

Priorita: Should have

Rozhraní aplikace by mělo být dostupné ve dvou režimech, ve světlém a tmavém. Uživatel by měl mít možnost v nastavení zvolit, zda chce používat světlý režim, tmavý režim, nebo režim dle systémového nastavení. Rozhraní aplikace by se pak mělo zobrazovat uživateli v tomto režimu.

1.3.1.20 FP-20 Push Notifikace

Priorita: Should have

Aplikace by měla být schopna uživateli posílat push notifikace. Tyto notifikace by měly uživatele upozorňovat v následujících situacích:

- Pokud se uživatel ještě neučil v daný den.
- Pokud se uživatel delší dobu v aplikaci nic neučil.
- Pokud uživateli končí předplatné aplikace a ještě ho neobnovil.
- Pokud uživatele začal sledovat jiný uživatel.
- Pokud bylo použito zmrazení streaku.
- Pokud byla zveřejněna aktivita jiného uživatele, kterého daný uživatel sleduje.
- Pokud se blíží konec týdne a uživateli hrozí propadnutí do nižší ligy.

V rámci této práce se budu zabývat pouze zprovozněním zobrazování push notifikací uživateli ve frontendové části aplikace. Podrobným fungováním odesílání push notifikací se bude zabývat kolega Bc. Jakub Vondráček jeho diplomové práci, která se zabývá backendovou částí této aplikace.

1.3.1.21 FP-21 Připomínání učení

Priorita: Could have

Uživatel by měl mít možnost si v aplikaci nastavit připomenutí, že se má v určitý čas učit. Aplikace by pak měla uživateli posílat push notifikace, které ho na učení upozorní, pokud se ještě v daný den neučil. Tato funkce je inspirována konkurenčními aplikacemi, které umožňují uživateli si nastavit každodenní připomenutí učení.

Dále by aplikace měla uživateli umožnit zapnout "chytré připomínání". Pokud uživatel tuto funkci zapne, nebude mu aplikace připomínat učení podle nastaveného času, ale na základě předchozího používání aplikace. Tato možnost by měla být v aplikaci nastavena jako výchozí.

1.3.1.22 FP-22 Systém zobrazování reklam

Priorita: Should have

Ve stávající aplikaci se reklamy zobrazují pouze po úspěšném dokončení učení a to pouze ve formě bannerové reklamy.

Nově by aplikace měla umožňovat zobrazování dalších typů reklam, v závislosti na zařízení uživatele. Na mobilních zařízeních by aplikace měla primárně zobrazovat reklamy přes celou obrazovku. Na desktopových zařízeních by pak aplikace měla zobrazovat reklamy v podobě bannerů. Reklamy by měla aplikace zobrazovat v následujících případech.

- Po skončení učení, ať už po úspěšném dokončení nebo po předčasném ukončení učení uživatelem.
- Pokud chce uživatel na účtu doplnit body energie.
- Pokud uživatel dostal odměnu s virtuální měnou a zvolil možnost dostání dvojnásobku odměny, pokud zhlédne reklamu.

Dále by aplikace měla uživateli v pravidelných intervalech zobrazovat reklamu na premium verzi aplikace.

Tento požadavek vychází z analýzy konkurenčních aplikací, kde většina aplikací zobrazuje reklamy v průběhu používání aplikace a zobrazuje reklamy na premium verzi aplikace.

1.3.1.23 FP-23 Nový systém předplatného

Priorita: Must have

Ve stávající aplikaci má uživatel možnost si zaplatit předplatné a tím získat přístup k pokročilejším funkcím. Jelikož v rámci této práce bude do aplikace přidána řada nových funkcí, je třeba tomu přizpůsobit i systém předplatného. Předplatné by nyní mělo být rozděleno do následujících kategorií.

- **Uživatelé bez předplatného** budou mít přístup ke všem základním kurzům a kariérním cestám v aplikaci. Budou ovšem omezeni systémem energie, který jim umožní projít pouze omezený počet lekcí a cvičení za den. V aplikaci budou těmto uživatelům zobrazovány reklamy a budou mít omezený přístup k chatbotovi. Nebudou moci získat certifikáty o dokončení kurzů.
- **Uživatelé s předplatným** budou mít neomezený přístup ke všem kurzům a kariérním cestám aplikace. Nebudou jim zobrazovány reklamy, budou moci využívat chatbota s nižšími omezeními a budou moci získávat certifikáty o dokončení kurzů.

V rámci této funkce bude dále třeba zajistit lepší uživatelskou zkušenost a informovanost uživatelů, které funkce jsou dostupné v premium verzi aplikace. Pokud uživatel například chce využít funkci, která je dostupná pouze uživatelům s předplatným, měla by mu být zobrazena stránka, která ho bude informovat, že je daná funkce dostupná pouze v premium verzi aplikace.

1.3.1.24 FP-24 Zkušební verze předplatného

Priorita: Should have

Aplikace by měla umožnit uživateli spustit zkušební verzi předplatného na určitou dobu zdarma. Tuto možnost by měli mít pouze uživatelé, kteří dříve zkušební verzi nikdy nevyužili.

1.3.1.25 FP-25 Přizpůsobení avatara uživatele

Priorita: Should have

Uživatel by měl mít možnost si přizpůsobit svého avatara, neboli postavu na profilovém obrázku, pomocí položek, které zakoupil v obchodě za virtuální měnu. V rámci tohoto požadavku bude také třeba upravit vzhled profilové stránky uživatele, aby byl profilový obrázek viditelnější. Pro přizpůsobení avatara by aplikace měla uživateli poskytnout samostatnou stránku, kde může změny provádět.

Uživatel by měl mít možnost si u avatara přizpůsobit barvu pleti, velikost těla, obličej, barvu očí, vlasy, barvu vlasů, doplňky, barvu brýlí, vousy, barvu vousů, pokrývku hlavy, oblečení, barvu oblečení a barvu pozadí profilového obrázku.

Tento požadavek vychází zejména z analýzy forem gamifikace a také analýzy konkurenční aplikace Duolingo, která podobnou funkci přizpůsobení si avatara uživatele nabízí.

1.3.1.26 FP-26 Rozšíření obchodu a odemykání položek

Priorita: Could have

V aplikaci již existuje stránka obchodu, v němž si uživatel může zakoupit pomocí virtuální měny profilové obrázky.

Vzhledem k novým požadavkům aplikace je třeba tento obchod rozšířit tak, aby bylo možné si zakoupit i položky, které jsou popsány v požadavku FP-25 Přizpůsobení avatara uživatele.

Dále by v obchodu měla být zavedena nová funkce odemykání položek. Některé položky by měly být dostupné ke koupi pouze pokud uživatel dosáhl určitého ocenění v aplikaci. Do té doby by se položky měly uživateli zobrazovat jako zamčené.

U zamknutých položek by měl uživatel mít možnost si zobrazit informace o tom, co musí splnit, aby danou položku odemknul.

1.3.1.27 FP-27 Uživatelský profil

Priorita: Should have

V aplikaci se již nachází stránka s uživatelským profilem, která obsahuje základní informace o daném uživateli. Po zavedení nových funkcí do aplikace je třeba tuto stránku aktualizovat tak, aby kromě již zobrazovaných informací o uživateli přehledně zobrazovala i následující informace.

- Avatar uživatele, popsáný v požadavku FP-25 Přizpůsobení avatara uživatele.
- Liga, ve které se uživatel nachází. Popis lig je popsán v požadavku FP-30 Žebříček uživatelů.
- Kurzy, které má uživatel zapsané a přehled, jaké úrovně v nich uživatel dosáhl.

Z důvodu přidání nových informací bude třeba změnit vzhled celé profilové stránky uživatele.

1.3.1.28 FP-28 Zmrazení streaku

Priorita: Should have

Aplikace uživateli zobrazuje tzv. streak, neboli skóre, které reprezentuje počet dní v řadě, v nichž uživatel splnil alespoň jedno učení.

Uživatel by nově měl mít v obchodě s virtuální měnou možnost si zakoupit do zásoby několik položek tzv. zmrazení streaku.

Pokud uživatel bude mít zakoupenou jednu nebo více těchto položek, a nesplní daný den žádné cvičení, jeho streak se tím nezruší a jedna tato položka se spotřebuje. Díky tomu bude uživatel moci zachovat svůj streak i když některý den například zapomene se učit.

1.3.1.29 FP-29 Oznámení o změně stavu streaku

Priorita: Should have

Pokud uživatel dokončí učení a jeho streak se zvýší na novou významnou hranici, například 100 dní, měla by mu aplikace zobrazit oznámení o dosažení této hranice.

Dále, pokud uživatel ztratí streak, tedy je mu resetován na 0 dní, měla by mu být tato informace oznámena po otevření aplikace.

1.3.1.30 FP-30 Žebříček uživatelů

Priorita: Could have

V aplikaci by měly být zavedeny tzv. ligy učení, které by měly fungovat následujícím způsobem. Začátkem každého týdne by měl být každý uživatel zařazen do skupiny s omezeným počtem uživatelů, kteří se nachází ve stejné lize, tedy získali předchozí týden podobné množství bodů XP. Tato skupina je reprezentována žebříčkem uživatelů, který je seřazen podle počtu XP, které získali v momentálním týdnu.

Na konci každého týdne by měl být uživatel přesunut do vyšší ligy, pokud se nacházel v žebříčku uživatelů na prvních několika pozicích. Pokud se uživatel nacházel v žebříčku na posledních několika pozicích, měl by být přesunut do nižší ligy. Pokud se uživatel nacházel v žebříčku mezi těmito pozicemi, měl by zůstat v aktuální lize.

Tento požadavek vychází z analýzy konkurenčních řešení, zejména z analýzy aplikací Mimo a Duolingo, v nichž jsou podobné ligy implementovány.

1.3.1.31 FP-31 Truhla s odměnou

Priorita: Should have

Po dokončení učení by uživateli měla být zobrazena stránka s truhlou, kterou bude moci uživatel otevřít. Po otevření truhly by měl uživatel získat náhodné množství virtuální měny, jako odměnu za jeho učení. Uživateli by následně mělo být nabídnuto, že může získat dvojnásobek této odměny, pokud zhlédne reklamu.

1.3.1.32 FP-32 Denní výzvy

Priorita: Could have

Aplikace by měla uživateli každý den zobrazit několik denních výzev, které může splnit pro získání truhly s odměnou (vizte požadavek FP-31 Truhla s odměnou). Tyto výzvy mají za cíl motivovat uživatele, aby se učil nad rámec jeho základního učení, aby vyzkoušel různé funkce aplikace, a aby si rozšířil svoje znalosti napříč různými kurzy a nejen v kurzech, které má zapsané.

V aplikaci by měly být následující typy denních výzev:

- Naučení se určitého počtu nových lekcí.

- Získání určitého počtu bodů XP.
- Učení se po určitý čas.
- Naučení se jedné lekce z určitého kurzu.
- Použití určité funkce aplikace, jako například zeptání se o radu chatbota nebo přizpůsobení avatara uživatele.

1.3.1.33 FP-33 Certifikáty o dokončení kurzu

Priorita: Should have

Po dokončení kurzu by měl mít uživatel možnost si vygenerovat a stáhnout certifikát o dokončení kurzu. Certifikát by měl být ve formátu PDF a měl by obsahovat základní informace o dokončeném kurzu a uživateli, který kurz dokončil.

1.3.1.34 FP-34 Sdílení pokroku mezi uživateli

Priorita: Should have

V aplikaci existuje základní funkce pro sdílení pokroku mezi uživateli. Tato funkce je ovšem příliš jednoduchá a s implementací nových funkcí je třeba ji přizpůsobit.

Aplikace by měla být přizpůsobena na sdílení následujících zpráv mezi uživateli.

- Když uživatel dosáhne určitého streaku.
- Když uživatel skončí na předních příčkách v lize.
- Když uživatel dosáhne určité úrovně v kurzu.
- Když uživatel dosáhne určitého ocenění.

1.3.1.35 FP-35 Odstranění přebytečných funkcí

Priorita: Must have

Jelikož se aplikace bude zaměřovat na výuku softwarového inženýrství, bude třeba odstranit funkce týkající se původní domény výuky jazyků. Mezi tyto funkce patří následující.

- Vyhledávání slovíček
- Přidávání slovíček do seznamu oblíbených slovíček
- Cvičení zaměřená na výuku jazyků a která nejsou využitelná pro výuku softwarového inženýrství

- Rozdělení typů lekcí dle aspektu výuky jazyků, například gramatické lekce, lekce slovní zásoby apod.
- Stránka slovíčka
- Výběr preferovaných témat kurzu. Tato funkce v původní aplikaci sloužila jako způsob přeskocení skupin lekcí slovíček, které se netýkaly zájmů a cílů uživatele. Uživatel tak například mohl vypnutím tématu "Příroda a životní prostředí" jednoduše ze svého učení vyřadit všechny lekce týkající se tohoto tématu. V nové aplikaci ovšem bude struktura kurzů a domény změněna natolik, že tato funkce již nebude relevantní a její ponechání by mohlo mít za následek značné navýšení komplexity celého systému.
- Přidávání lekcí do seznamu oblíbených lekcí. Žádná z analyzovaných aplikací tuto funkci nenabízí. Funkce v rámci původní domény výuky jazyků mohla být užitečná, nicméně nyní by zbytečně mohla komplikovat uživatelské rozhraní.
- Odebrání stránky lekce. Tato stránka dříve zobrazovala například seznam vyučovaných slovíček v dané lekci. Zároveň bylo možné na této stránce přidávat lekci do seznamu oblíbených lekcí. V rámci domény výuky softwarového inženýrství žádná z analyzovaných aplikací samostatnou stránku pro lekci neobsahuje. Nyní ovšem stránka žádné výhody nemá a pouze komplikuje uživatelské rozhraní.
- Vytváření vlastních lekcí slovíček. Původní aplikace umožňovala uživatelům seskupovat slovíčka v aplikaci do vlastních lekcí. Tato funkce již není relevantní z důvodu nové domény a struktury kurzů.

1.3.2 Nefunkční požadavky

1.3.2.1 NP-1 Nový design aplikace

Priorita: Should have

Jelikož aplikace mění své zaměření z výuky jazyků na výuku softwarového inženýrství, je třeba změnit a přizpůsobit i samotný vzhled aplikace.

Měl by být vytvořen nový design uživatelského rozhraní, který bude aplikován jak na nové funkce, tak na funkce existující.

Základní komponenty používané při implementaci by dále měly být extrahovány do samostatné knihovny tak, aby bylo možné je využít i v dalších částech projektu, jako například na webových stránkách.

Tato knihovna komponentů by měla stavět na již existujících komponentech a měla by využívat již zavedené technologie, tedy knihovny React, TypeScript, Storybook a Material UI.

1.3.2.2 NP-2 Aktualizace kódu aplikace

Priorita: Should have

V kódu aplikace by měly být provedeny následující změny se záměrem zvýšit jeho kvalitu a udržitelnost a zjednodušit budoucí vývoj.

- Aktualizace používaných knihoven, nástrojů a jejich konfigurací.
- Aktualizace používaných formaterů a linterů.
- Dockerizace frontendové části aplikace.
- Zavedení pravidel a konvencí pro nástroje, které využívají umělou inteligenci pro generaci kódu, dokumentačních komentářů a testů, s cílem podpoření vývoje aplikace a zvýšení efektivity práce.
- Odstranění existujícího mocku API, místo kterého by měla aplikace využívat testovací prostředí poskytované backendovou částí aplikace.

Tento požadavek byl vytvořen na základě výstupů analýzy existujícího řešení.

1.3.2.3 NP-3 Zavedení automatizovaných testů

Priorita: Should have

Do frontendové části aplikace by měly být zavedeny automatizované testy s cílem zajištění správného fungování aplikace a zvýšení její kvality. Automatizované testy by se měly zaměřit na klíčové části aplikace, kterými jsou proces učení, registrace a přihlášení uživatele, výběr a zápis kurzů, a kupování předplatného.

Tento požadavek vychází jednak z analýzy existujícího řešení a jednak z návrhu úprav do budoucna v mé bakalářské práci [1] zabývající se zmíněným řešením.

1.3.2.4 NP-4 Automaticky generované třídy API

Priorita: Should have

V rámci kódu frontendu by měly být automaticky generovány třídy reprezentující backendové API na základě OpenAPI specifikace [14] poskytované backendovou částí aplikace.

Požadavek vychází z analýzy existujícího řešení. Jeho cílem je zjednodušení implementace a zajištění konzistence mezi frontendovou a backendovou částí aplikace.

1.3.2.5 NP-5 Animace a efekty

Priorita: Should have

Do aplikace by měly být přidány základní animace s cílem zlepšit uživatelskou zkušenost a udržet pozornost uživatelů. Uživatelské rozhraní by tak mělo působit vizuálně poutavějším dojmem a mělo by výrazněji reagovat na interakce s uživatelem.

Tento požadavek vychází zejména z analýzy konkurenčních řešení, ve které konkurenční aplikace jako Mimo nebo Scrimba využívají animace a efekty.

1.3.2.6 NP-6 Sestavování a nasazování PWA

Priorita: Should have

Aplikace by měla být přizpůsobena tak, aby bylo možné ji sestavit a nasa-
dit jako PWA, neboli progresivní webovou aplikaci. Tomu by měly být přizpů-
sobený i devops procesy, v rámci kterých by měla být aplikace automaticky
sestavována a nasazována jako PWA.



Kapitola 2

Návrh

Výstupem předchozí kapitoly je seznam jednotlivých požadavků, které je třeba v aplikaci implementovat. Cílem této kapitoly bude z těchto požadavků vytvořit návrh a specifikaci jednotlivých funkcí a vytvořit tak podrobné podklady pro samotný vývoj aplikace.

Jelikož aplikace zásadně mění svoje zaměření a bude se zaměřovat na jinou doménu, je třeba tomu přizpůsobit celé uživatelské rozhraní. Proto se nejprve zaměřím na návrh samotného vzhledu uživatelského rozhraní včetně výběru palety barev, písma, vzhledu komponentů, animací a efektů. Tímto návrhem se budu následně řídit při tvorbě *high fidelity* prototypu nového uživatelského rozhraní. Nakonec se zaměřím na podrobnou specifikaci jednotlivých funkcí aplikace s využitím metodiky Behavior-Driven Development a jazyku Gherkin.

Díky detailnímu návrhu uživatelského rozhraní a podrobné specifikaci jednotlivých funkcí tak bude aplikace připravena na samotný vývoj, kterému se pak budu věnovat v další kapitole.

2.1 Návrh vzhledu uživatelského rozhraní

V této sekci se zaměřím na samotný vzhled uživatelského rozhraní a na volbu barev, písma, ikon a vhodných animací. Cílem je vytvořit konzistentní vzhled, který bude následně možné použít v prototypu a samotné aplikaci.

Jelikož stávající aplikace využívá knihovnu Material UI, která implementuje standard Material Design 2 [8], budu celý návrh zakládat na tomto standardu. Zmíněný standard klade důraz na konzistentní využívání palety barev, typografické hierarchie a komponentů. Dále se také zaměřuje na přístupnost uživatelského rozhraní, podporu světlého i tmavého režimu a dodržování doporučených osvědčených při návrhu uživatelského rozhraní [15].

Na tomto místě je vhodné zmínit, že v současné době již existuje nová verze tohoto standardu Material Design 3, nicméně v době psaní tohoto textu nejsou k dispozici žádné knihovny, které by tento standard implementovaly v takové

míře, aby je bylo možné pro tento projekt vhodně využít. Proto budu nadále pro aplikaci využívat knihovnu Material UI, která se řídí standardem Material Design 2.

2.1.1 Principy návrhu

Před samotnou tvorbou návrhu je třeba stanovit základní principy, kterými se bude nový vzhled uživatelského rozhraní řídit. Při sestavování těchto principů budu vycházet z několika zdrojů. Zprvce budu vycházet z analýzy existujících řešení, které jsem se věnoval v předchozí kapitole.

Zadruhé budu vycházet z toho, jakým způsobem by měla aplikace působit na uživatele. Aplikace by například měla působit jako důvěryhodný zdroj informací, aby uživatel byl ochotný se pomocí ní vzdělávat.

Zatřetí jsem se rozhodl zaměřit na návrh s ohledem na emocionální potřeby uživatelů [16, s. 385]. Emoční potřeby uživatelů mohou být při návrhu určitých typů systémů klíčové a mohou aplikaci zásadně odlišit od konkurenčních řešení [16, s. 387]. Pokud například navrhujeme aplikaci pro děti, můžeme se cíleně zaměřit na návrh uživatelského rozhraní, které bude barevné, interaktivní a zábavné na používání. U jiných typů systémů, jako například v aplikaci obchodní burzy, by podobný způsob designu pravděpodobně působil dětinsky, neprofesionálně a nedůvěryhodně. Jelikož se v rámci této práce zaměřuji na tvorbu vzdělávací aplikace, mohou být emoční potřeby, jako motivace, pocit pokroku a úspěchu klíčové a proto na nich budu zakládat návrh uživatelského rozhraní.

Na základě zmíněných zdrojů jsem identifikoval tyto základní principy, kterými se budu při návrhu řídit.

- **Minimalismus** – Hlavním cílem aplikace je učít uživatele programování. Při takové činnosti je ovšem po uživateli vyžadována vysoká úroveň pozornosti a koncentrace. Ve srovnání s ostatními aplikacemi tak může být používání této aplikace značně kognitivně náročné. Proto jsem se rozhodl při návrhu zaměřit na minimalistický design. Design by tak měl co nejvíce omezit zbytečné informace a zjednodušit prvky uživatelského rozhraní tak, aby nepřehlcovaly uživatele a neodváděly jeho pozornost od samotného učení. Uživatelské rozhraní by tak mělo svým minimalismem pomoci uživateli při učení a používání aplikace.
- **Důvěryhodnost** – Jelikož se aplikace zaměřuje na výuku a předávání informací uživatelům, je naprosto klíčové, aby působila jako důvěryhodný zdroj informací. Uživatelské rozhraní by tak mělo působit moderně, důvěryhodně a profesionálně.
- **Zábava** – Jedním z cílů aplikace je motivovat uživatele k dlouhodobému učení a používání aplikace. Proto by pro uživatele by mělo být učení a používání aplikace zábavné a měl by se chtít k aplikaci dlouhodobě vracet.

- **Pokrok a úspěch** – Při používání aplikace a zejména při učení by měl mít uživatel pocit, že dělá pravidelně pokrok a že se posouvá blíže ke svému cíli. Aplikace by tak měla uživateli často ukazovat jeho pokroky a úspěchy.

V následujících sekcích na základě těchto charakteristik vyberu konkrétní barvy, typografii, ikony a animace.

2.1.2 Výběr palety barev

Jako první krok návrhu vzhledu jsem se rozhodl zvolit paletu barev¹. Vybrané barvy pak budu konzistentně využívat napříč prototypem a celou aplikací. Barvy budu volit v souladu se standardem Material Design 2 a budu volit barvy jak pro světlý, tak pro tmavý režim uživatelského rozhraní. Při volbě palety barev se také budu řídit standardem přístupnosti *Web Content Accessibility Guidelines 2.2* [17], díky kterému by mělo být uživatelské rozhraní dobře použitelné napříč různými zařízeními a také použitelné osobami se zdravotním postižením, jako například osobami se sníženým barvocitem.

2.1.2.1 Primární barva

První barvu, kterou budu volit je primární barva. Tato barva reprezentuje samotnou značku aplikace a je napříč aplikací nejčastěji používána [18]. Využívá se zejména ke zvýraznění důležitých prvků, kterými jsou například tlačítka nebo navigační lišty.

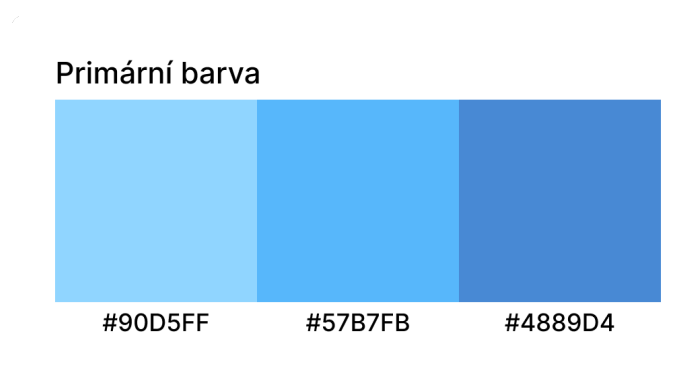
Barvu jsem se rozhodl zvolit v souladu se stanovenými principy tak, aby aplikace působila důvěryhodně a profesionálně. Proto jsem jako primární barvu zvolil odstín modré, protože bývá tato barva považována za barvu důvěryhodnosti, inteligence a přesnosti [19].

Jako základ pro volbu primární barvy jsem vybral světle modrou s kódem #90D5FF. Tato barva by měla evokovat pocit důvěryhodnosti a klidu [20]. Jelikož se jedná o světlou barvu, zvolil jsem následně její dvě tmavší varianty #57B7FB a #4889D4. Pro volbu těchto variant jsem využil oficiální nástroj Material palette generator², který umožňuje pro danou barvu vygenerovat různé světlé varianty. Dále jsem pomocí tohoto nástroje vybral kontrastní barvu textu #000000, která bude používána pro text zobrazovaný na primární barvě. Vybraná černá barva textu dodržuje s primární barvou kontrastní poměr 4.5:1 dle AA standardu přístupnosti *Web Content Accessibility Guidelines 2.2* [17]. Díky tomu bude text na primární barvě dostatečně kontrastní a bude tedy snadno čitelný.

Všechny tyto barvy generovány zmíněným nástrojem jsou v souladu se standardy přístupnosti UI [18]. Výslednou volbu primární barvy a jednotlivých variant lze nalézt na obrázku 2.1.

¹Dostupné na: <https://www.figma.com/design/VZFuqWojrXq8yWjrBIq4OH>

²Dostupné na: <https://m2.material.io/design/color/the-color-system.html>



■ Obrázek 2.1 Volba primární barvy

2.1.2.2 Sekundární barva

Material Design 2 umožňuje kromě primární barvy také definovat sekundární barvu, která slouží k doplnění primární barvy, vytvoření vizuálního kontrastu a zvýraznění určitých prvků rozhraní. Na rozdíl od primární barvy, která typicky reprezentuje samotnou značku aplikace a je v rozhraní přítomna na většině míst, je sekundární barva používána střídměji a pouze u některých prvků UI. Prakticky se tak využívá například na zvýraznění odkazů, v přepínačích, indikátorech stavů aplikace nebo v tzv. *floating action buttons*. Přestože Material Design 2 definuje tuto sekundární barvu, její volba a použití jsou volitelné. [18]

Při volbě sekundární barvy jsem vycházel z principů teorie barev a ze standardně používaných barevných schémat, která definují rozložení barev na barevném kruhu [21]. Tato schémata slouží jako praktický nástroj pro posouzení, které barvy se vhodně doplňují tak, aby bylo možné je spolu využít například v uživatelském rozhraní. Na základě těchto schémat je tak možné zvolit sekundární barvu, která bude vhodně doplňovat primární barvu a vytvářet tak vizuálně příjemný vzhled uživatelského rozhraní. Při výběru sekundární barvy jsem volil z následujících schémat.

Monochromatické schéma Monochromatické schéma využívá jeden odstín a jeho světlejší či tmavší varianty. V rozhraní obvykle působí klidně a kompaktně. Je také snadné na použití, protože designéři musí pracovat pouze s jedním hlavním odstínem. [21]

Analogické schéma Analogické schéma kombinuje sousední barvy na barevném kruhu. Toto schéma je často možné pozorovat v přírodě, kde se podobné odstíny barev často vyskytují blízko sebe. Výsledkem bývá harmonický, méně kontrastní vzhled a přírodně vypadající vzhled. [22]

Komplementární schéma Komplementární schéma využívá dvojici protilehlých barev na barevném kruhu. Vytváří tak vysoký kontrast, což je přínosné pro uchycení pozornosti uživatele na důležité prvky UI. Toto schéma tak umožňuje využití teplejších a chladnějších odstínů v UI. [22]

Rozděleně komplementární schéma Rozděleně komplementární schéma vychází z komplementární dvojice odstínů, ale místo přímého protikladu jsou využity dvě sousední barvy k doplňkové barvě. V praxi tak designéři mají možnost využít více barev a paleta tak může působit vyváženěji. [22] [21]

Triadické schéma Triadické schéma pracuje se třemi barvami rovnoměrně rozmístěnými na barevném kruhu do tvaru rovnostranného trojúhelníku. Schéma může působit živěji a dynamičtěji, nicméně stále je třeba určit která ze tří barev bude v UI dominantní a které budou používány střídavě. [22]

Tetradické schéma Tetradické schéma využívá čtyři barvy tvořící na barevném kruhu obdélník, často ve formě dvou komplementárních dvojic. Nabízí široké možnosti kombinací, avšak jeho použití v designu UI může být náročné, zejména kvůli udržení konzistence ve využití jednotlivých barev. [22]

Čtvercové schéma Čtvercové schéma pracuje, jako tetradické schéma, se čtyřmi barvami rovnoměrně rozmístěnými na kruhu. Podobně jako tetradické schéma poskytuje velkou variabilitu, nicméně zvyšuje riziko vizuální roztržitosti, pokud nejsou stanoveny a dodržována jasná pravidla pro použití jednotlivých barev. Oproti tetradickému schématu umožňuje čtvercové schéma sestavit vyrovnanější vzhled ve srovnání s vysokým kontrastem, který tetradické schéma často vytváří. [22]

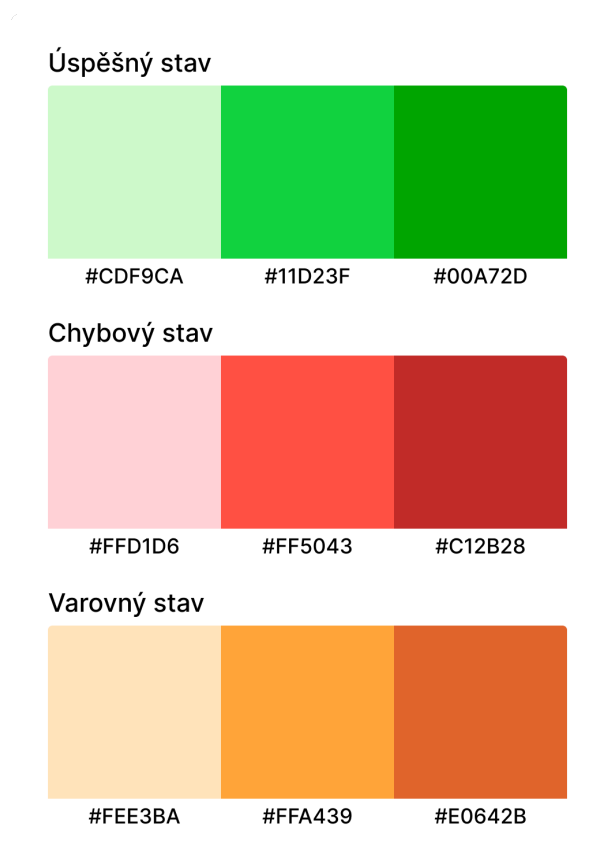
V rámci výběru sekundární barvy jsem se nakonec rozhodl využít monochromatické schéma, tedy pracovat primárně s jedním odstínem modré a jeho variantami, a samostatnou sekundární barvu pro rozhraní nedefinovat. Toto rozhodnutí přímo plyne ze stanovených principů minimalismu a důvěryhodnosti. Díky omezení počtu barev by tak mělo uživatelské rozhraní působit minimalističtěji a mělo by být snadnější dodržet konzistentní využití barev napříč návrhem a aplikací. Celý design by tak měl působit profesionálnějším a čistším dojmem. Zároveň, jelikož jsem jako primární barvu zvolil světle modrou, mělo by výsledné rozhraní působit klidným a méně rušivým dojmem [20]. To bude podstatné zejména při samotném učení, kdy je uživatel vystaven vyšší kognitivní zátěži a příliš agresivní barvy by mohly být rozptylující.

Přestože jsem se rozhodl pro monochromatické schéma a využívat tak hlavně primární barvu, bude aplikace obsahovat řadu dalších barev, které budou používány například pro indikaci stavů v aplikaci, jako například pro indikaci správných a špatných odpovědí uživatele. Tomu se budu věnovat v následujících sekcích.

2.1.2.3 Stavové barvy

Kromě zvolení primární a sekundární barvy je třeba také zvolit barvy pro indikaci různých stavů, které mohou v aplikaci nastat. Tyto barvy budou například použity pro indikaci správných a špatných odpovědí uživatele, nebo pro indikaci chyby a varování v aplikaci. Jako základní odstíny jsem zvolil zelenou pro indikaci úspěšných stavů, oranžovou pro indikaci varování a červenou pro indikaci chyb. Jelikož budou tyto barvy často používány při učení, rozhodl jsem se vybrat pastelové varianty těchto barev. Podobně jako primární barva by tak měly působit klidně, příjemně a neměly by příliš rozptylovat uživatele při učení.

Podobně jako u primární barvy, umožňuje Material Design 2 definovat hlavní variantu **main**, světlou variantu **light** a tmavou variantu **dark** pro každou barvu. Dále knihovna Material UI umožňuje definovat také odpovídající barvu textu **contrastText** [23], která je dostatečně kontrastní vůči dané barvě a dodržuje tak standard přístupnosti *Web Content Accessibility Guidelines 2.2*, dále WCAG 2.2 [17]. Pro každý stav jsem tak vybral varianty barev, které lze nalézt na obrázku 2.2.



■ Obrázek 2.2 Volba barev pro indikaci stavů

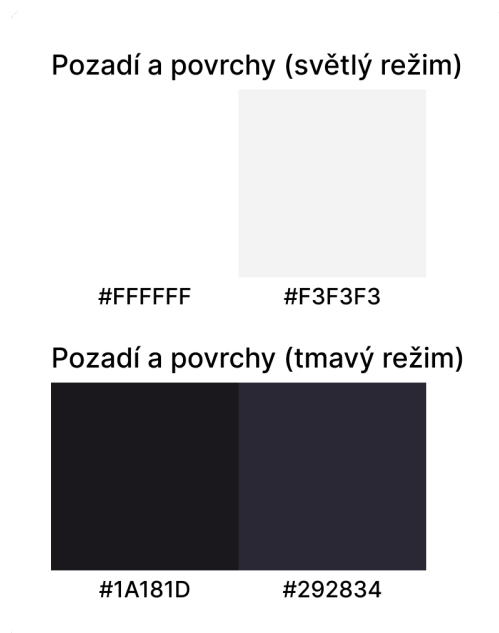
Podobně jako u primární barvy jsem pro volbu jednotlivých variant využil nástroj Material palette generator.

2.1.2.4 Barvy pozadí

Co se týče tmavého režimu, je třeba tuto barvu upravit tak, aby byla dostatečně kontrastní vůči pozadí dle standardu přístupnosti AA *Web Content Accessibility Guidelines 2.2* [17]. U primární barvy je tak třeba snížit její saturaci, aby byl dodržen kontrastní poměr 4.5:1 vůči pozadí při používání této barvy u textových prvků.

Jako barvu pozadí **background** jsem se rozhodl využít výchozí barvu Material Design 2, tedy #FFFFFF. Tato barva je používána zejména pro pozadí jednotlivých stránek. Jako barvu povrchů, neboli barvu pozadí některých komponentů, jsem dále zvolil světle šedou barvu #F3F3F3 označovanou jako **paper**, díky které bude možné komponenty lépe odlišit od pozadí stránky. Tyto barvy budou používány ve světlém režimu aplikace.

Ukázky barev pozadí a povrchů lze nalézt na obrázku 2.3.



■ **Obrázek 2.3** Volba barev pozadí a povrchů

2.1.2.5 Barvy textu

Pro textové prvky jsem se rozhodl využít výchozí barvu Material Design 2, tedy černou barvu #000000. Kromě této primární barvy textu knihovna Material UI ještě umožňuje definovat sekundární barvu textu, kterou je možné využít

například pro odstavce a vytvořit tak silnější vizuální kontrast mezi nadpisy a normálním textem. Jako sekundární barvu jsem se tak rozhodl využít barvu #6f6f6f, která je výrazně světlejší než černá barva, nicméně i pro běžný text stále splňuje kontrastní poměr mezi textem a barvami pozadí 4.5:1 dle AA standardu WCAG 2.2 [17].

2.1.2.6 Tmavý režim

Jedním z definovaných požadavků je dostupnost tmavého režimu aplikace. Pro vytvoření tmavého režimu je třeba jednotlivé barvy upravit tak, aby je bylo možné využívat na tmavých pozadích a stále odpovídaly standardu WCAG 2.2 [17]. Při tvorbě tmavého režimu jsem postupoval dle doporučení oficiální dokumentace Material Design 2 [24].

Jako první krok jsem se rozhodl zvolit samotné barvy pozadí a povrchů pro tmavý režim. Jako barvu pozadí jsem zvolil tmavě šedou barvu #1A181D a jako barvu povrchů jsem dále zvolil tmavě šedou barvu #292834. Hlavním důvodem pro použití tmavě šedých barev, místo čistě černé barvy, je snaha dosáhnout nižšího kontrastu mezi světlým textem a tmavým pozadím. Text je pak lépe čitelný a méně namáhá oči uživatele. [24]

Po vybrání barvy pozadí je třeba přizpůsobit saturaci primární barvy, barvy stavů a barvy textu tak, aby byl dodržen kontrastní poměr 4.5:1 mezi textem a tmavým pozadím [17]. Pro nalezení odpovídajících desaturovaných barev jsem využil nástroj Colour contrast checker³, který mi umožnil vybrat k jednotlivým barvám jejich desaturované varianty a zkontrolovat dodržení kontrastního poměru.

Nakonec jsem vybral barvy textu pro tmavý režim. Jako primární barvu jsem zvolil výchozí bílou barvu #FFFFFF a jako sekundární barvu jsem zvolil #989898. Obě barvy opět splňují požadovaný kontrastní poměr.

2.1.3 Typografie

V této sekci se zaměřím na volbu písma. Podobně jako u volby palety barev budu vycházet ze standardu Material Design 2 a budu volit vlastnosti písma v souladu se standardem přístupnosti WCAG 2.2 [17].

2.1.3.1 Výběr písma

Jako základní písmo jsem se rozhodl využít rodinu písem Inter. Tato rodina písem byla publikována v roce 2017 zaměřuje se na moderní a čistý vzhled, podporuje přes 147 jazyků a je široce využívána napříč různými obory. Pro tuto rodinu písem jsem se rozhodl zejména z důvodu moderního a profesionálního vzhledu a její široké podpory a rozšířenosti. [25]

³Dostupné na: <https://colourcontrast.cc/>

Při volbě písma je možné dále využít tzv. párování fontů [13, s. 271-272], při kterém se pro nadpisy využije jiná rodina písma, než pro běžný text. Díky tomu lze dosáhnout výraznějšího rozlišení mezi nadpisy a běžným textem a také zajímavějšího vzhledu rozhraní. Při tomto párování je ovšem třeba použít písma, která jsou od sebe dostatečně odlišná a využít tak například patková a bezpatková písma. Pro zachování minimalismu jsem se rozhodl toto párování nevyužít a použít tak rodinu písem Inter pro nadpisy i běžný text.

Dále, jelikož budou v aplikaci často zobrazovány ukázky kódu, bylo potřeba zvolit písmo pro zobrazování kódu. Pro tento účel jsem se rozhodl využít rodinu písem JetBrains Mono⁴, která je speciálně navržena tak, aby byla dobře použitelná v editorech kódu a aby byl samotný kód dobře čitelný. [26]

2.1.3.2 Hierarchie a škálování

Po volbě rodiny písma je třeba zvolit typografickou škálu, která určuje velikosti jednotlivých textových prvků rozhraní, jako například velikosti nadpisů, podnadpisů a odstavců. Prvním krokem při volbě škály je zvolení základní velikosti písma, například 16 pixelů. Tato velikost reprezentuje velikost běžného textu v odstavcích. Následně, na základě zvolené škály, se toto číslo násobí daným poměrem, čímž se získají velikosti nadpisů, podnadpisů a dalších textových prvků.

Volba typografické škály tak určuje, jaký je rozdíl mezi velikostmi jednotlivých textových prvků. Tento rozdíl pak ovlivňuje, jak velký bude vizuální kontrast mezi jednotlivými prvky. Škály s vysokým kontrastem, jako například *Golden ratio* s poměrem 1,618, jsou vhodné pro velké obrazovky [27]. Naopak škály s nízkým kontrastem, jako například *Major Second* s poměrem 1,125, umožňují zobrazit větší množství textu na jedné stránce, jelikož jednotlivé nadpisy nezabírají tolik prostoru. Takové škály se často používají například na mobilních zařízeních, kde je prostor na obrazovce omezený.

Při volbě škály jsem nejprve použil oficiální nástroj pro generaci typografické škály dle standardu Material Design 2 [28]. Po aplikaci této škály při návrhu uživatelského rozhraní jsem ovšem došel k názoru, že tato škála vytváří příliš vysoký kontrast mezi textovými prvky. Velké nadpisy zabírají příliš mnoho prostoru a jsou při návrhu rozhraní pro tento typ aplikace takřka nepoužitelné. Nedostatek menších nadpisů tak neumožňuje dosáhnout detailnějšího odlišení jednotlivých sekcí a podsekcí, což značně znepráhledňuje například stránky se cvičeními a vysvětlením látky. Došel jsem tak k závěru, že výchozí oficiální typografická škála Material Design 2 není pro tuto aplikaci vhodná.

Z tohoto důvodu jsem se rozhodl zvolit vlastní typografickou škálu, která bude vhodnější pro tento typ aplikace. Rozhodl jsem se vybrat dvě škály, jednu pro mobilní zařízení a jednu pro desktopová zařízení. Pro mobilní zařízení jsem se rozhodl využít škálu s nižším kontrastem *Minor Third* s poměrem 1.200

⁴Dostupné na: <https://www.jetbrains.com/lp/mono/>

a pro desktopová zařízení škálu *Major Third* s poměrem 1,250. Díky těmto škálám tak bude možné plně využít všechny velikosti nadpisů a text tak bude přizpůsoben jednotlivým zařízením. [27]

Pro generaci jednotlivých velikostí písma jsem následně využil nástroj *Typescale*⁵. Při aplikaci vygenerované škály v návrhu rozhraní jsem následně dodržoval standardy *Material Design 2*, které se týkají vlastností jednotlivých textových prvků a definují například doporučenou velikost písma.

2.1.4 Ikony a ilustrace

V rámci návrhu rozhraní je dále třeba zvolit vhodnou kolekci ikon a ilustrací. Pro vzhled ikon jsem se rozhodl využít kolekci *Material Icons*⁶. Pro návrh jsem se rozhodl využít zaoblenou variantu těchto ikon, aby rozhraní působilo hravějším a zábavnějším dojmem.

Jelikož bude aplikace obsahovat různé formy gamifikace a dalších motivačních prvků, je třeba do návrhu zahrnout kromě ikon i různé ilustrace. Jako zdroj ilustrací jsem se rozhodl využít repozitář ilustrací publikovaných pod svobodnými licencemi *SVG Repo*⁷. Jednotlivé ilustrace jsem se pak snažil volit tak, aby měly vzájemně konzistentní styl.

2.1.5 Animace a zvukové efekty

V rámci návrhu jsem se rozhodl v souladu s nefunkčními požadavky navrhnout i animace a efekty, které by měly být při implementaci použity. Příslušné animace a efekty jsem vyznačil poznámkami u jednotlivých stránek v návrhu uživatelského rozhraní.

Mezi použité animace patří například animace přechodu mezi stránkami, animace zmáčknutí tlačítka, animace konfet při dokončení lekce nebo získání ocenění, animace indikátoru pokroku při učení a další.

Kuriozitou při návrhu rozhraní bylo využití zvukových efektů. Zvukové efekty bývají ve webových aplikacích využívány velice zřídka, zejména kvůli tomu, že nejsou pro interakci s uživatelem potřeba a zbytečně by uživatele rušily při používání aplikace. V jistých případech mohou být ovšem zvukové efekty vhodné pro podpoření určitých emocí u uživatele a pro zvýšení intenzityžitku z vykonání určitých akcí. V této aplikaci budu například využívat zvukové efekty při samotném učení uživatele pro indikaci správných a špatných odpovědí. Jako zdroj zvukových efektů jsem se rozhodl využít soubor efektů *Material Product Sounds*⁸. Podobně jako u animací jsem vyznačil zvukové efekty v návrhu uživatelského rozhraní na místech, kde by měly být použity. [29]

⁵Dostupné na: <https://typescale.com/>

⁶Dostupné na: <https://fonts.google.com/icons>

⁷Dostupné na: <https://www.svgrepo.com/>

⁸Dostupné na: <https://m2.material.io/design/sound/sound-resources.html>

2.2 Návrh prototypu uživatelského rozhraní

V této sekci se zaměřím na návrh *high fidelity* prototypu uživatelského rozhraní⁹, tedy prototypu, který bude obsahovat detailní návrh uživatelského rozhraní včetně definovaných stylů, barev, písma, ilustrací a interakcí [30, s. 53-54]. Cílem je vytvořit návrh uživatelského rozhraní, které bude přesně specifikovat vzhled aplikace a přechody mezi jednotlivými stránkami. Prototyp tak bude sloužit jako podklad pro samotnou implementaci aplikace.

Pro tvorbu prototypu jsem zvolil nástroj Figma¹⁰, protože umožňuje rychlé iterace návrhu, práci s komponentami a také propojení obrazovek do podoby interaktivního prototypu. Při návrhu jsem vycházel ze stylů definovaných v předchozí sekci.

2.2.1 Tvorba prototypu

V rámci mé bakalářské práce jsem se mimo jiné věnoval vytvoření návrhu uživatelského rozhraní pro stávající aplikaci na výuku angličtiny [1]. V průběhu let jsem ovšem zpozoroval několik nedostatků tohoto prototypu. Použitá knihovna předdefinovaných komponentů je zbytečně složitá a komplikuje vytváření a používání nových komponentů. Prototyp také nevyužívá moderní funkce programu Figma, jako například definice proměnných komponentů a globálních proměnných. S ohledem na nově definované styly, nové funkce a na nový účel aplikace jsem se tak rozhodl vytvořit celý návrh uživatelského rozhraní od začátku a bez návaznosti na existující prototyp.

Při tvorbě prototypu jsem postupoval následovně. Nejprve jsem zadefinoval pomocí proměnných jednotlivé styly dle návrhu v předchozí sekci. Následně jsem provedl hrubý návrh základních komponent a stránek aplikace. Poté jsem se v rámci agilního procesu vývoje vždy zaměřil na jednu funkci, kterou jsem navrhl podrobněji a zprovoznil její interakce v prototypu. Po dokončení návrhu dané funkce jsem provedl její implementaci a testování, které jsou popsány v následujících kapitolách. Po dokončení dané funkce jsem se přesunul na následující funkci, dle její priority. Po dokončení návrhu všech funkcí jsem provedl závěrečné úpravy návrhu a organizaci stránek a komponentů.

Při návrhu jsem kladl důraz na znovupoužitelnost komponent a na využití pokročilejších možností Figmy, díky kterým je možné prototyp v budoucnu jednodušeji rozšiřovat a upravovat. Konkrétně jsem široce využíval komponenty a jejich varianty, díky kterým je zajištěna vyšší konzistence samotného návrhu. Dále jsem využíval lokální proměnné komponentů a globální kolekce proměnných, díky kterým je možné kdykoliv měnit styly v návrhu, aniž by bylo nutné modifikovat samotné stránky a komponenty [31].

Jednotlivé komponenty jsem dále strukturoval tak, aby bylo možné je přímo implementovat s využitím jejich zadaných parametrů v prototypu. Při

⁹Dostupné na: <https://www.figma.com/design/VZFuqWojrXq8yWjrBIq4OH>

¹⁰Dostupné na: <https://www.figma.com/>

návrhu jsem také využíval funkci *auto layout*, díky které je možné přesně specifikovat rozložení komponentů tak, aby byly responzivní a použitelné v různých částech UI [32].

Součástí návrhu jsou také dva režimy aplikace, světlý a tmavý. Oba režimy sdílí stejné rozvržení stránek a komponentů a liší se pouze aplikovanými styly, zejména barvami pozadí, textu a stavů. Pro definici světlého a tmavého režimu jsem využil funkci módů proměnných [33], díky které je možné pro každý režim definovat jiné hodnoty proměnných a následně snadno mezi tmavým a světlým režimem přepínat. Díky tomu je opět zajištěna vyšší konzistence návrhu, jelikož není třeba pro světlý a tmavý režim navrhovat samostatné stránky, ale je možné pouze přepínat mezi aplikovanými styly.

Vytvořený návrh vzhledu jsem také doplnil o interakce reprezentující reálné používání aplikace. Zaměřil jsem se zejména na navigaci mezi obrazovkami tak, aby bylo možné procházet základní scénáře používání aplikace. Díky tomu bylo možné prototyp interaktivně procházet bez samotné implementace a provádět na něm základní testování.

Pro ukázkou návrhu jsem vybral několik stránek z prototypu. Tyto stránky lze najít na obrázcích 2.4, 2.5, 2.6 a 2.7.

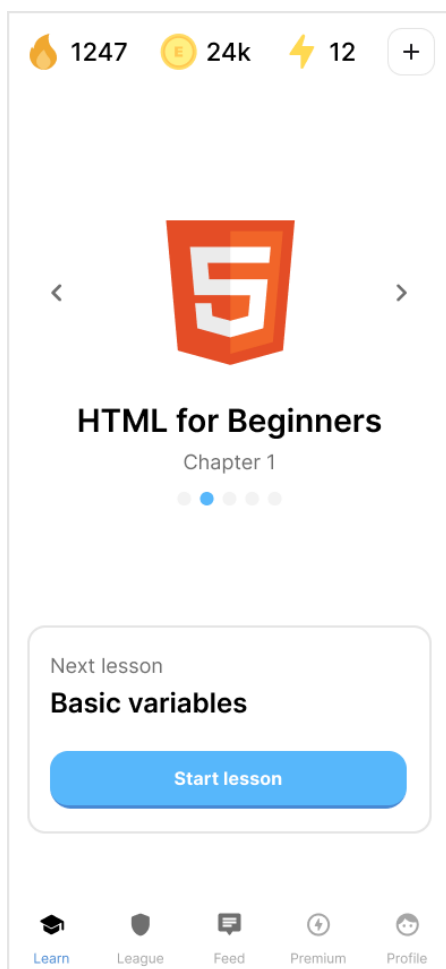
2.2.2 Responsivita

Návrh prototypu jsem realizoval přístupem *mobile-first* [34]. Primární cílovou platformou této aplikace jsou mobilní zařízení, proto jsem nejprve navrhl vzhled stránek pro menší rozlišení a teprve následně jsem řešil úpravy pro větší displeje. Tento postup mi umožnil soustředit se na efektivní využití prostoru a na jednoduché rozvržení stránek, které je na malých obrazovkách potřeba.

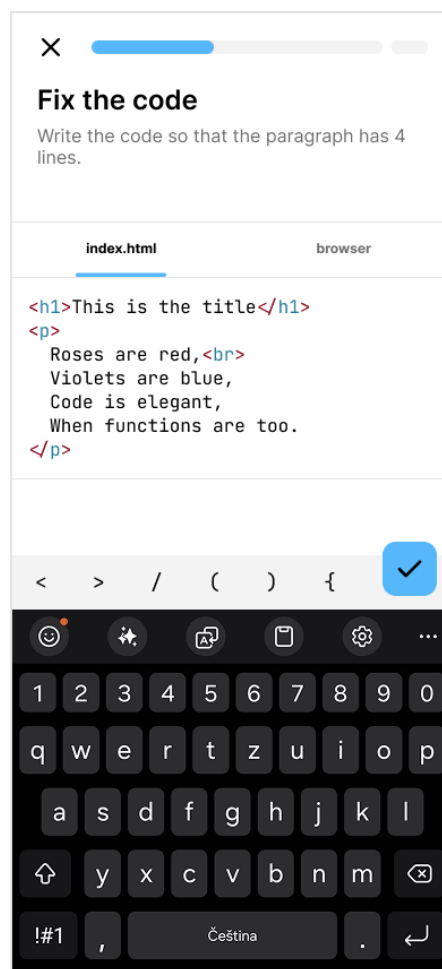
Po dokončení návrhu prototypu bylo třeba zvolit, jakým způsobem bude rozhraní responzivní, tedy jak se bude rozložení stránek přizpůsobovat různým velikostem displejů. Při návrhu responzivity je možné využít několik standardních rozložení stránek.

Rozložení *mostly fluid* například typicky pracuje s bloky obsahu, které se přesouvají pod sebe se zmenšujícím se rozlišením [35, s. 7]. Dále je možné využít rozložení *column drop*, které využívá několik sloupců s obsahem a se zmenšujícím se rozlišením se sloupce opět přesouvají pod sebe [35, s. 8]. Také je možné využít rozložení *layout shifter*, které představuje nejkomplexnější ze zmíněných přístupů a spočívá v navržení jiné struktury stránek pro každou velikost displeje [35, s. 9].

V tomto projektu jsem se rozhodl využít přístup *tiny tweaks*, který spočívá v uspořádání veškerého obsahu stránky do jediného sloupce a zachování tohoto rozložení stránky napříč všemi zařízeními [35, s. 10]. Na stránkách jsou tak při změně velikosti displeje provedeny pouze drobné úpravy. Implementace se tak bude řídit primárně návrhem pro mobilní zařízení a na větších displejích se budou měnit především maximální šířka obsahu, odsazení a práce s volným prostorem. Kromě toho bude struktura stránek zachována a bude



■ **Obrázek 2.4** Ukázka návrhu hlavní stránky aplikace



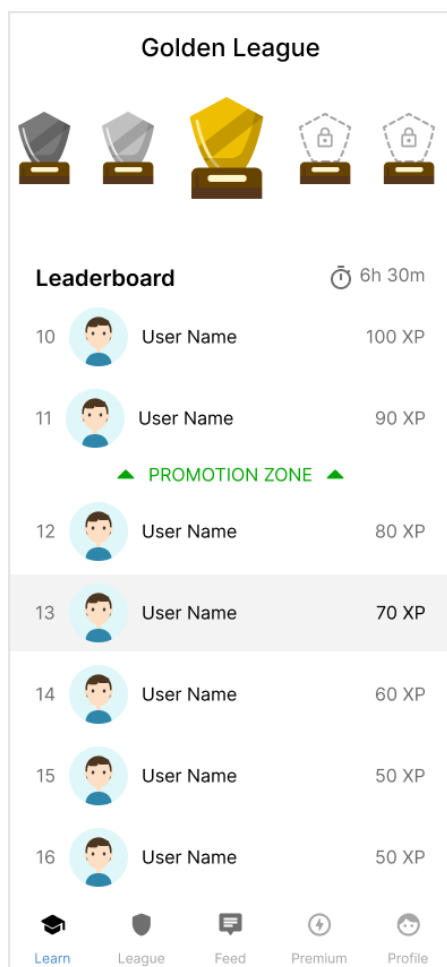
■ **Obrázek 2.5** Ukázka návrhu cvičení programování

napříč zařízeními totožná.

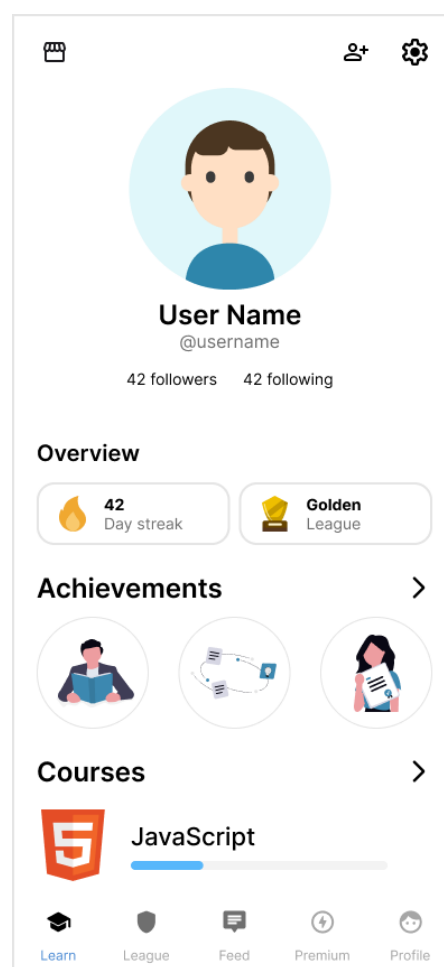
Jako důsledek zvolení tohoto přístupu je tak možné udržet návrh jednoduchý a konzistentní. Zároveň díky totožnému rozložení stránek je sníženo riziko rozdílného chování napříč různými velikostmi displejů. Aplikaci by tak mělo být snazší rozšiřovat, udržovat a testovat.

2.2.3 Testování prototypu

Po dokončení počáteční verze návrhu prototypu jsem provedl heuristickou analýzu, jejímž cílem bylo zachytit základní problémy v použitelnosti a konzistenci návrhu. Díky této analýze bylo možné identifikovat nedostatky návrhu ještě před samotnou implementací a upravit návrh tak, aby lépe odpovídal očekávanému chování uživatelů a byl v souladu se základními principy a heuristikami



Obrázek 2.6 Ukázka návrhu žebříčku uživatelů



Obrázek 2.7 Ukázka návrhu uživatelského profilu

návrhu. [36]

Heuristická analýza a její výsledky jsou podrobněji popsány v kapitole testování. Na základě poznatků z této analýzy jsem následně upravil návrh tak, aby byl v souladu s heuristikami a byla tak vznikající aplikace pro uživatele lépe použitelná.

2.3 Návrh funkcí aplikace

V této sekci se budu věnovat podrobnější specifikaci funkcí navrhované aplikace. Nejprve se zaměřím na popis jednotlivých aktérů, kteří budou s aplikací interagovat. Následně na základě funkčních požadavků sepišu seznam případů užití, které budou podrobněji specifikovat, jak mají jednotlivé části aplikace fungovat z pohledu uživatele. Nakonec na základě případů užití sepišu podrob-

nou specifikaci vybraných funkcí v jazyce Gherkin. Tato specifikace umožní efektivní spouštění automatizovaných testů pomocí nástroje Cucumber a bude také sloužit jako dokumentace toho, jak se aplikace skutečně chová. Mimo jiné bude možné tuto specifikaci využít také jako zadání pro AI agenty, kteří mohou v budoucnu na základě této specifikace přesně implementovat požadované funkce, nebo například generovat dokumentaci a testy.

2.3.1 Popis aktérů

V rámci specifikace funkcí je třeba popsat tzv. aktéry, kteří reprezentují role, které mohou s aplikací interagovat a vykonávat v ní určité akce. Tito aktéři budou využiti jak v popisu případů užití, tak ve specifikaci funkcí pomocí jazyka Gherkin. Na základě požadavků aplikace jsem identifikoval následující aktéry.

2.3.1.1 Uživatel

Uživatel je osoba, která využívá aplikaci k učení se programování. Mezi jeho hlavní aktivity patří například zapisování si kurzů, učení se programování, interakce s ostatními uživateli a interakce s prvky gamifikace v aplikaci.

2.3.1.2 Premium uživatel

Premium uživatel je osoba, která má aktivní předplatné nebo zkušební verzi předplatného. Premium uživatel může využívat všechny funkce jako uživatel. Kromě těchto akcí může vykonávat i další akce, ke kterým nemá běžný uživatel přístup.

2.3.1.3 Korektor

Korektor je osoba zodpovědná za kontrolu kvality výukového obsahu, aby byl správný a srozumitelný. V rámci aplikace může používat funkce podobně jako premium uživatel. Kromě toho může využívat podrobnější nahlašování chyb při učení.

2.3.2 Specifikace případů užití

Jako další krok při návrhu funkcí jsem se rozhodl sepsat případy užití. Případy užití představují způsob, jak specifikovat funkční chování systému z pohledu jeho aktérů. Každý případ užití popisuje konkrétní cíl aktéra a interakce se systémem, které k dosažení tohoto cíle vedou. Díky tomuto přístupu je tak možné detailněji formulovat požadavky a identifikovat okrajové případy, které je třeba při návrhu a implementaci ošetřit. Případy užití tak mohou být užitečné pro vývojáře, protože jim poskytnou detailnější popis toho, jak má daná

část systému fungovat. Dále mohou být využity testery například jako podklad pro vytváření automatizovaných testů. Díky tomu, že bývají případy užití specifikovány pomocí přirozeného jazyka, mohou být čitelné i pro osoby, které nejsou technicky zaměřené. Celkově tak tvoří přesnější specifikaci systému, na základě které lze následně postupovat při návrhu a implementaci samotné aplikace. [37]

V průběhu let se vyvinulo několik způsobů, jak k případům užití přistupovat a jak je specifikovat. Základy techniky specifikace případů užití položil Ivar Jacobson v roce 1986. Alistair Cockburn [38] následně popsal metodiku pro psaní podrobnějších případů užití. Přesto však prosazoval názor, že by forma specifikace případů užití měla odpovídat potřebám a rozsahu konkrétního projektu. Další techniku specifikace případů užití také popsal například Martin Fowler, který prosazoval názor, že neexistuje jeden standardní způsob, jak specifikovat případy užití, a že formát a rozsah případů užití by měl být zvolen dle potřeb konkrétního projektu. [30, s. 22-27]

V této práci jsem se tak rozhodl vycházet z práce Alistaira Cockburna [38] a specifikovat případy užití způsobem, který bude užitečný pro vývoj této konkrétní aplikace. Pro každý případ užití tak budu specifikovat jeho identifikátor, název, popis, hlavního aktéra, hlavní scénář a případné alternativní scénáře.

Pro specifikaci scénářů jsem se rozhodl využít jazyk Gherkin, který přináší několik zásadních výhod, kterým se budu věnovat v následující sekci. Jelikož se v jazyce Gherkin definují scénáře přímo v repozitáři aplikace v souborech s příponou `.feature` [39], rozhodl jsem se nejprve v rámci specifikace případů užití popsat hlavní a alternativní scénáře pouze stručným popisem a detailně je pak specifikovat přímo v repozitáři aplikace pomocí jazyka Gherkin. Díky tomu budu moci nejprve získat přehled o všech scénářích a okrajových případech a až poté provést jejich podrobnější specifikaci. Výsledkem tak bude využití jak výhod specifikace případů užití, tak i výhod jazyka Gherkin, kterému se budu věnovat v následující sekci.

V rámci specifikace případů užití jsem tak sestavil následující seznam případů užití.

2.3.2.1 UC-X Name of the use case

Popis:	Use case description
Hlavní aktér:	Actor name
Hlavní scénář:	Main scenario description
Alternativní scénář 1:	Alternative scenario description
Alternativní scénář 2:	Alternative scenario description

2.3.3 Specifikace funkcí pomocí jazyka Gherkin

Po sepsání případů užití jsem se rozhodl nové funkce přesněji specifikovat pomocí jazyka Gherkin [39]. Gherkin je jednoduchý strukturovaný jazyk, který

umožňuje pomocí přirozeného jazyka popsat chování systému v podobě textových scénářů. Díky přesně dané struktuře je následně možné tyto scénáře strojově zpracovat pomocí nástroje Cucumber¹¹ a ověřit, že se aplikace chová v souladu se specifikací pomocí automatizovaných testů. Celý proces je postavený na přístupu *Behavior-Driven Development* (BDD) [40], tedy na metodice vývoje, při které se přesně specifikuje očekávané chování systému a následně se vytvoří automatizované testy, které ověří, že se aplikace skutečně podle specifikace chová. Tento přístup vznikl jako reakce na rozšířenou metodiku *Test-Driven Development* (TDD) s cílem pomoci vývojářům a testerům nahlížet na testování více z pohledu samotného chování aplikace, přesněji pochopit a definovat požadované chování a minimalizovat tak nejednoznačnosti v zadání a implementaci jednotlivých funkcí [41].

Tento přístup má několik zásadních výhod. Zaprvé samotná specifikace slouží jako přesná dokumentace toho, jak se současný systém skutečně chová. Díky přirozenému jazyku může být užitečná nejen testerům a vývojářům, ale i lidem, kteří nejsou technicky zaměřeni. Zadruhé díky automatizovaným testům, které jsou spouštěny dle jednotlivých scénářů, je možné průběžně ověřovat, že se aplikace skutečně chová přesně v souladu se specifikací. Zatřetí přesné specifikace funkcí mohou sloužit jako podklady pro asistenty umělé inteligence, kteří pomáhají vývojářům a testerům s implementací kódu. Používání nástrojů umělé inteligence tak může být díky přesné specifikaci a automatizovaným testům výrazně efektivnější. [39]

Z výše zmíněných důvodů jsem se rozhodl využít jazyk Gherkin a nástroj Cucumber pro specifikaci funkcí a automatizaci testů s cílem zajistit efektivnější vývoj a kvalitnější implementaci samotných funkcí.

2.3.3.1 Postup při specifikaci

Funkce jsem specifikoval na základě definovaných případů užití a v souladu s dokumentací nástroje Cucumber [39], tedy přímo v repozitáři frontendu aplikace v souborech s příponou `.feature`. Tyto soubory jsem následně využil jako podklad pro implementaci a dále pro automatizaci testů, kterou jsem popsal v kapitole testování.

Soubory v jazyce Gherkin obsahují definici funkce **Feature** a jeden nebo více scénářů **Scenario** s jednotlivými kroky. Kroky scénáře jsou definovány pomocí klíčových slov **Given** a **When**, které definují předpoklady a dále

Soubor ve formátu Gherkin typicky obsahuje definici funkce (*Feature*), jeden nebo více scénářů (*Scenario*) a jednotlivé kroky. Kroky jsou definovány pomocí klíčových slov, kterými jsou například slova **Given** definující předpoklady, **When** definující událost v systému, **Then** definující očekávaný výsledek a **And** umožňující definovat více kroků stejného typu. Příklad definice funkce pomocí jazyka Gherkin lze nalézt v ukázce kódu 1.

¹¹Dostupné na: <https://cucumber.io/>

Feature: Vyhodnocení odpovědi ve cvičení

Scenario: Uživatel odpoví správně

Given uživatel je přihlášen

And má otevřené cvičení

When odešle správnou odpověď

Then aplikace zobrazí pozitivní zpětnou vazbu

And přičte uživateli body

■ **Výpis kódu 1** Ukázka specifikace funkce pomocí jazyka Gherkin

Implementace

3.1 Postup při realizaci projektu

3.1.1 Příprava implementace

3.1.1.1 Aktualizace konvencí a linterů

Na začátku přípravy implementace jsem se zaměřil na zavedení nových konvencí a linterů s cílem zvýšit kvalitu kódu a zajistit jeho konzistenci napříč celým projektem. V projektu byly již zavedeny knihovny ESLint a Prettier, nicméně jejich konfigurace byla zastaralá a neodpovídala současným standardům. Proto jsem aktualizoval obě knihovny, jejich konfigurace a zavedl jsem nové konvence a pravidla pro psaní kódu. Po zavedení těchto změn jsem provedl refaktoring stávajícího kódu, aby odpovídal novým pravidlům.

3.1.1.2 Dockerizace

Následně jsem provedl dockerizaci vývojového prostředí, která do té doby nebyla v aplikaci zavedena. Cílem bylo zajistit, aby veškerý vývoj probíhal v konzistentním prostředí a aby se zjednodušilo nastavování aplikace a práce s jednotlivými příkazy, skripty a knihovnami.

3.1.1.3 Zavedení nástrojů umělé inteligence

3.1.1.4 Refaktoring zastaralých komponent

3.1.1.5 Design System

Po zavedení nových konvencí a linterů jsem se přesunul k vytvoření design systému. Protože aplikace již využívala knihovnu Material UI, rozhodl jsem se na této knihovně stavět a vytvořit na jejím základě samostatnou knihovnu komponentů s názvem Eduriam UI. Tato knihovna bude kromě této aplikace

využívána i v dalších částech projektu, jako jsou například webové stránky nebo další interní programy.

Knihovnu jsem vytvořil jako samostatný repozitář, který jsem následně vydal jako npm balíček ve službě GitHub Packages. Díky tomu je možné knihovnu snadno verzovat a distribuovat napříč různými částmi projektu. Tuto knihovnu jsem následně importoval do frontendové části aplikace, kde jsem provedl rozsáhlý refaktoring kódu frontendu, při kterém jsem přesunul komponenty do nově vytvořené knihovny a upravil jejich implementaci tak, aby odpovídala zavedeným konvencím.

V rámci knihovny Eduriam UI jsem použil jazyk TypeScript a knihovny React a Material UI. Dále jsem zavedl knihovnu Storybook, která umožňuje snadné prohlížení, dokumentaci a testování jednotlivých komponentů. Do knihovny jsem také zavedl nástroje Prettier a ESLint pro zajištění konzistentního formátování a zvýšení kvality kódu.

..... Kapitola 4

Testování

Závěr



Příloha A

Nějaká příloha

Sem přijde to, co nepatří do hlavní části.

Bibliografie

1. JENÍČEK, Jan. *Frontend webové aplikace na učení se angličtiny* [online]. 2024. [cit. 2026-01-14]. Dostupné z: <https://dspace.cvut.cz/entities/publication/0b0498b3-d721-4ee3-9f61-83da8c90de4f>. Bakalářská práce. České vysoké učení technické, Fakulta informačních technologií.
2. MIMO. *Mimo* [online]. [cit. 2026-01-14]. Dostupné z: <https://mimo.org/>.
3. DETERDING, Sebastian; DIXON, Dan; KHALED, Rilla; NACKE, Lenart. From game design elements to gamefulness: defining “gamification”. In: *Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments*. Tampere, Finland: Association for Computing Machinery, 2011, s. 9–15. MindTrek '11. ISBN 9781450308168. Dostupné z DOI: 10.1145/2181037.2181040.
4. CODECADEMY. *Catalog* [online]. [cit. 2026-01-14]. Dostupné z: <https://www.codecademy.com/catalog>.
5. CODECADEMY. *Partnerships* [online]. [cit. 2026-01-14]. Dostupné z: <https://www.codecademy.com/pages/partnerships>.
6. SOLOLEARN. *Sololearn* [online]. [cit. 2026-01-14]. Dostupné z: <https://www.sololearn.com/>.
7. GOOGLE. *Applying density* [online]. [cit. 2026-01-14]. Dostupné z: <https://m2.material.io/design/layout/applying-density.html>.
8. MUI. *Material UI* [online]. [cit. 2026-01-14]. Dostupné z: <https://mui.com/material-ui/>.
9. SCRIMBA. *Courses* [online]. [cit. 2026-01-14]. Dostupné z: <https://scrimba.com/courses>.
10. GOOGLE. *System font or web-safe font* [online]. [cit. 2026-01-14]. Dostupné z: https://fonts.google.com/knowledge/glossary/system_font_web_safe_font.

11. HLAVÁČ, Jan. *Backend webové aplikace na učení se angličtiny* [online]. 2024. [cit. 2026-01-14]. Dostupné z: <https://dspace.cvut.cz/entities/publication/ecf8a20f-9558-45d3-b91d-8ea9affc8d09>. Bakalářská práce. České vysoké učení technické, Fakulta informačních technologií.
12. HUDAIB, Amjad; MASADEH, Raja; HAJ QASEM, Mais; ALZAQEBAH, Abdullah. Requirements Prioritization Techniques Comparison. *Modern Applied Science*. 2018, roč. 12, č. 2, s. 64–65. ISSN 1913-1844. Dostupné z DOI: 10.5539/mas.v12n2p62.
13. TIDWELL, Jenifer; BREWER, Charles; VALENCIA, Aynne. *Designing Interfaces: Patterns for Effective Interaction Design*. 3. vyd. Sebastopol, CA: O'Reilly Media, Inc., 2020. ISBN 978-1-492-05196-1.
14. SWAGGER. *OpenAPI Specification* [online]. [cit. 2026-01-14]. Dostupné z: <https://swagger.io/specification/>.
15. GOOGLE. *Material Design 2* [online]. [cit. 2026-01-21]. Dostupné z: <https://m2.material.io/>.
16. HARTSON, Rex; PYLA, Pardha. *The UX Book: Agile UX Design for a Quality User Experience*. 2. vyd. Cambridge, MA: Morgan Kaufmann, 2019. ISBN 978-0-12-805342-3.
17. W3. *Web Content Accessibility Guidelines (WCAG) 2.2* [online]. 2024. [cit. 2026-01-22]. Dostupné z: <https://www.w3.org/TR/WCAG22>.
18. GOOGLE. *The color system* [online]. [cit. 2026-01-21]. Dostupné z: <https://m2.material.io/design/color/the-color-system.html>.
19. PAVLÍČEK, Josef. *Colors* [online]. 2025. [cit. 2026-01-21]. Dostupné z: https://docs.google.com/presentation/d/1l_T7H8KyT43RTaGzcsLXo4t1w08s30aD6yLaMZZ1InQ.
20. FIGMA. *Light blue* [online]. 2026. [cit. 2026-01-22]. Dostupné z: <https://www.figma.com/resource-library/what-is-color-theory/>.
21. FIGMA. *What is color theory?* [online]. 2026. [cit. 2026-01-22]. Dostupné z: <https://www.figma.com/resource-library/what-is-color-theory/>.
22. FIGMA. *Color Wheel* [online]. 2026. [cit. 2026-01-22]. Dostupné z: <https://www.figma.com/color-wheel/>.
23. MATERIAL UI. *Palette* [online]. 2026. [cit. 2026-01-22]. Dostupné z: <https://mui.com/material-ui/customization/palette/>.
24. GOOGLE. *Dark theme* [online]. 2026. [cit. 2026-01-22]. Dostupné z: <https://m2.material.io/design/color/dark-theme.html>.
25. ANDERSSON, Rasmus. *The Inter typeface family* [online]. 2026. [cit. 2026-01-23]. Dostupné z: <https://rsms.me/inter/>.
26. JETBRAINS. *JetBrains Mono* [online]. 2026. [cit. 2026-01-28]. Dostupné z: <https://www.jetbrains.com/lp/mono/>.

27. ANAND, Surya. *Typographic Scales* [online]. 2025. [cit. 2026-01-28]. Dostupné z: <https://designcode.io/typographic-scales>.
28. GOOGLE. *The type system* [online]. 2026. [cit. 2026-01-23]. Dostupné z: <https://m2.material.io/design/typography/the-type-system.html>.
29. GOOGLE. *Applying sound to UI* [online]. 2026. [cit. 2026-01-23]. Dostupné z: <https://m2.material.io/design/sound/applying-sound-to-ui.html>.
30. PAVLÍČEK, Josef. *UI design steps* [online]. 2025. [cit. 2026-01-29]. Dostupné z: <https://docs.google.com/presentation/d/1ClDShyC8D8cN2LGrqUVJ2U5eEKn5z9MpkFPmzwZX4Zc>.
31. FIGMA. *Guide to variables in Figma* [online]. 2026. [cit. 2026-01-29]. Dostupné z: <https://help.figma.com/hc/en-us/articles/15339657135383-Guide-to-variables-in-Figma>.
32. FIGMA. *Guide to auto layout* [online]. 2026. [cit. 2026-01-29]. Dostupné z: <https://help.figma.com/hc/en-us/articles/360040451373-Guide-to-auto-layout>.
33. FIGMA. *Modes for variables* [online]. 2026. [cit. 2026-01-29]. Dostupné z: <https://help.figma.com/hc/en-us/articles/15343816063383-Modes-for-variables>.
34. TALARICO, Donna. What is mobile-first design? Why does it matter? *Recruiting & Retaining Adult Learners*. 2019, roč. 22, č. 2, s. 3–3. ISSN 2155-644X. Dostupné z DOI: <https://doi.org/10.1002/nsr.30533>.
35. PAVLÍČEK, Josef. *Responsive design* [online]. 2025. [cit. 2026-01-29]. Dostupné z: <https://docs.google.com/presentation/d/1IAILjefRAXMu9vQ3qWQzDgaQCqWYbVLnc77j9cqz1XU>.
36. NIELSEN, Jakob. Enhancing the explanatory power of usability heuristics. In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. Boston, Massachusetts, USA: Association for Computing Machinery, 1994, s. 152–158. CHI '94. ISBN 0897916506. Dostupné z DOI: 10.1145/191666.191729.
37. MLEJNEK, Jiří. *Analýza a sběr požadavků* [online]. 2026. [cit. 2026-02-01]. Dostupné z: https://moodle-vyuka.cvut.cz/pluginfile.php/819387/mod_resource/content/7/03.prednaska.pdf.
38. COCKBURN, Alistair. *Writing Effective Use Cases*. 1. vyd. Addison-Wesley Professional, 2000. ISBN 9780321605795.
39. THE CUCUMBER OPEN SOURCE PROJECT. *Introduction* [online]. 2026. [cit. 2026-02-01]. Dostupné z: <https://cucumber.io/docs/>.

40. THE CUCUMBER OPEN SOURCE PROJECT. *Behaviour-Driven Development* [online]. 2025. [cit. 2026-02-01]. Dostupné z: <https://cucumber.io/docs/bdd/>.
41. THE CUCUMBER OPEN SOURCE PROJECT. *History of BDD* [online]. 2025. [cit. 2026-02-01]. Dostupné z: <https://cucumber.io/docs/bdd/history>.

Obsah příloh

/	
└─ readme.txt.....	stručný popis obsahu média
└─ exe.....	adresář se spustitelnou formou implementace
└─ src	
└─ impl.....	zdrojové kódy implementace
└─ thesis.....	zdrojová forma práce ve formátu $\text{\LaTeX}$
└─ text.....	text práce
└─ thesis.pdf.....	text práce ve formátu PDF